

Imię i nazwisko	Kierunek	Grupa	Temat
Szymon Maślanka	Edukacja Techniczno-Informatyczna	2	Internetowa strona kina

1. Cel ćwiczenia

Głównym celem ćwiczenia było zaprojektowanie i zaimplementowanie w pełni funkcjonalnego systemu webowego do zarządzania kinem, działającego w architekturze Full-stack. Cele szczegółowe obejmowały:

- Praktyczne zastosowanie architektury klient-serwer: Zrozumienie podziału ról między backendem (serwerem aplikacji) a frontendem (interfejsem użytkownika) oraz ich komunikacji za pomocą protokołu HTTP
- Implementacja relacyjnej bazy danych: Zaprojektowanie struktury tabel w MySQL, która efektywnie przechowuje dane o filmach, seansach i rezerwacjach, oraz opanowanie języka SQL w praktyce (tworzenie tabel, relacji i zapytań).
- Zarządzanie bezpieczeństwem danych wrażliwych: Nabycie umiejętności bezpiecznego przechowywania kluczy API, haseł i danych dostępowych do bazy danych przy użyciu zmiennych środowiskowych (mechanizm .env).
- Opanowanie autoryzacji bezstanowej (Stateless Auth): Implementacja mechanizmu JSON Web Token (JWT), co pozwala na bezpieczne logowanie użytkowników bez konieczności przechowywania sesji po stronie serwera.
- Integracja asynchronicznej komunikacji: Wykorzystanie biblioteki Axios do pobierania danych w czasie rzeczywistym, co pozwala na stworzenie dynamicznego interfejsu (np. aktualizacja dostępnych miejsc na sali kinowej bez odświeżania całej strony).
- Zarządzanie zależnościami projektu: Poznanie środowiska Node.js i menedżera pakietów npm, służącego do instalacji i konfiguracji zewnętrznych modułów (Express, MySQL2, Dotenv).

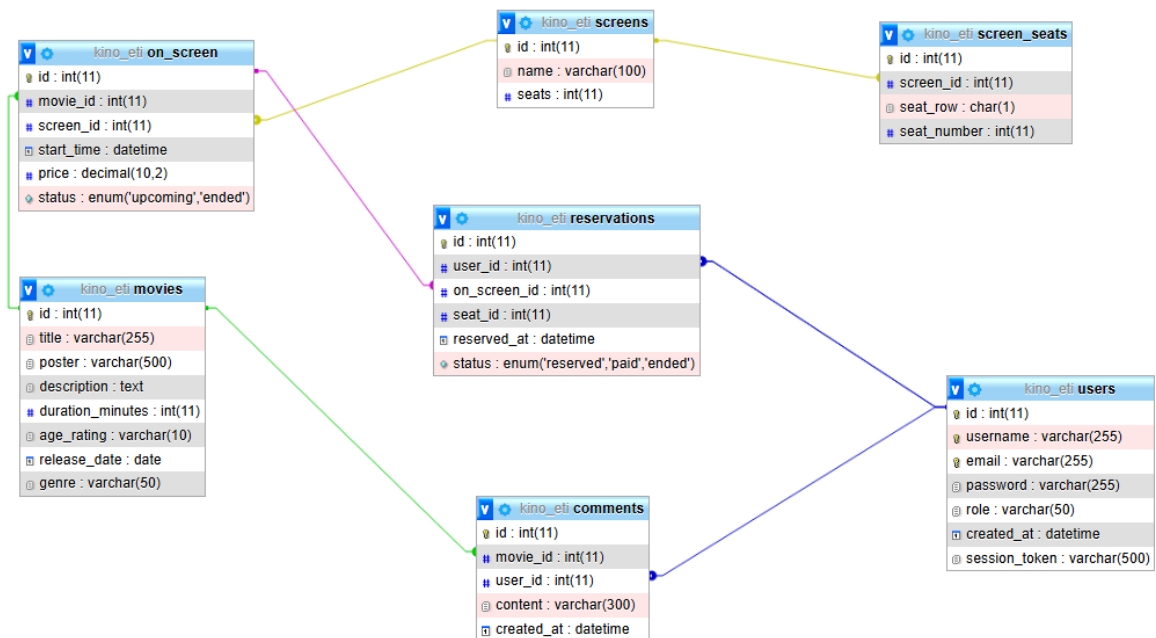
2. Baza danych

2.1 Wstęp

Baza danych została zrealizowana w modelu relacyjnym w środowisku MariaDB. Struktura bazy opiera się na 7 powiązanych tabelach, które kompleksowo obsługują procesy kinowe. Unikalnym rozwiązaniem jest zaimplementowana procedura składowana `generate_seats_for_screen`, która dynamicznie generuje układ współrzędnych foteli (rzędy i numery) dla nowo tworzonych sal. System relacji (Foreign Keys) zapewnia integralność danych, a zastosowanie ról użytkowników (Enum: admin/user)

oraz statusów rezerwacji pozwala na precyzyjne zarządzanie procesem sprzedaży biletów

2.2 Opis Tabeli ich Roli



Rys 1. Struktura bazy

Strukturę bazy można podzielić na trzy logiczne obszary:

A. Zarządzanie Zasobami (Sale i Miejsca)

- **screens (Sale)**: Przechowuje informacje o fizycznych salach kinowych (nazwa, ogólna liczba miejsc).
- **screen_seats (Miejsca)**: Najbardziej szczegółowa tabela fizyczna. Każdy rekord to konkretny fotel (rzęd, numer) przypisany do konkretnej sali.
- **Procedura generate_seats_for_screen**: To zaawansowany element bazy. Automatyzuje proces tworzenia układu sali – zamiast wpisywać ręcznie 100 miejsc, procedura sama wylicza rzędy (A, B, C...) i numery, a następnie wstawia je do tabeli screen_seats.

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `generate_seats_for_screen`
(IN `screenId` INT) BEGIN
    DECLARE total INT;
    DECLARE seatIndex INT DEFAULT 0;
    DECLARE seatRow CHAR(1);
    DECLARE seatNumber INT;

    -- Pobranie liczby miejsc dla sali
    SELECT seats INTO total FROM screens WHERE id = screenId;
```

```

WHILE seatIndex < total DO
    SET seatRow = CHAR(65 + (seatIndex DIV 12)); -- A, B, C, ...
    SET seatNumber = (seatIndex MOD 12) + 1;      -- 1-12

    INSERT INTO screen_seats (screen_id, seat_row, seat_number)
    VALUES (screenId, seatRow, seatNumber);

    SET seatIndex = seatIndex + 1;
END WHILE;
END$$

DELIMITER ;

```

Rys 2. Zastosowanie procedury w kodzie

B. Repertuar i Treści (Filmy i Seanse)

- **movies (Filmy):** Katalog filmów zawierający tytuły, opisy, plakaty, czas trwania oraz kategorię wiekową.
- **on_screen (Seanse):** Tabela łącząca. Wiąże film z konkretną salą o konkretnej godzinie. Zawiera również cenę biletu i status (czy seans już się odbył).
- **Comments (Interakcja):** Pozwalają użytkownikom oceniać filmy lub dyskutować z użytkownikami.

C. Użytkownicy i Sprzedaż

- **users (Użytkownicy):** Przechowuje dane dostępowe. Zastosowano tu podział na role (user, admin). Pole session_token przechowuje token JWT danego użytkownika.
- **reservations (Rezerwacje):** Kluczowa tabela sprzedażowa. Łączy użytkownika z konkretnym seansem i konkretnym miejscem na sali. Posiada system statusów (reserved, paid, ended).

2.3 Inne mechanizmy bazy

- **Integralność danych:** Używa ON DELETE CASCADE w tabeli rezerwacji i komentarzy. Oznacza to, że jeśli użytkownik usunie swoje konto, baza automatycznie "posprząta" i usunie jego komentarze oraz rezerwacje, zapobiegając powstawaniu tzw. rekordów-sierot.

```

ALTER TABLE `comments`
    ADD CONSTRAINT `comments_ibfk_1` FOREIGN KEY (`movie_id`) REFERENCES
`movies` (`id`),
    ADD CONSTRAINT `comments_ibfk_2` FOREIGN KEY (`user_id`) REFERENCES
`users` (`id`) ON DELETE CASCADE;

```

Rys 3. Automatyzacja usuwania połączonych danych

- **Optymalizacja (Indeksy):** Każda tabela posiada klucze główne oraz indeksy na kluczach obcych (ADD KEY). Dzięki temu wyszukiwanie np. wszystkich

rezerwacji dla danego filmu odbywa się błyskawicznie, nawet przy tysiącach rekordów.

3 Technologie oraz zasady strony

3.1 Wstęp i Architektura Systemu

Projekt "Kino ETI" to aplikacja webowa służąca do zarządzania repertuarem kinowym oraz systemem rezerwacji biletów. System został zaprojektowany w architekturze klient-serwer, z wyraźnym rozdzieleniem Frontendu oraz Backendu dla przejrzystości. Poniżej przedstawiona struktura plików w folderze głównym.

```
+---backend
|   | .env
|   | cron.js
|   | db.js
|   | package-lock.json
|   | package.json
|   | server.js
|   | tmdb.js
|   |
|   +---middleware
|   |   adminOnly.js
|   |   authMiddleware.js
|   |
|   +---node_modules
|   \---routes
|       admin_control.js
|       auth.js
|       movies.js
|       reservations.js
|       users.js
|
\---frontend
    +---css
    |   admin_panel.css
    |   biblioteka.css
    |   biblioteka_bez.css
    |   client.css
    |   client_settings.css
    |   index.css
    |   movies.css
    |   movies_bez.css
    |   reservation.css
    |   user_edit.css
    |   user_reservations.css
    |
    +---html
    |   admin_panel.html
    |   biblioteka.html
    |   biblioteka_bez.html
    |   client.html
    |   client_settings.html
    |   index.html
    |   movies.html
    |   movies_bez.html
    |   reservation.html
    |   user_edit.html
    |   user_reservations.html
    |
```

```

+---img
|   baner.jpg
|   logo.ico
|   logo.png
|
\---js
    admin_panel.js
    biblioteka.js
    biblioteka_bez.js
    client.js
    client_settings.js
    index.js
    movies_web.js
    movies_web_bez.js
    reservation.js
    user_edit.js
    user_reservations.js

```

Rys 4. Struktura plików

Zastosowana struktura plików opiera się na separacji logiki backendowej od frontendowej. Katalog backend został podzielony na moduły: routes (obsługa endpointów API), middleware (kontrola dostępu i bezpieczeństwo) oraz usługi pomocnicze jak cron i tmdb. Warstwa frontendu zachowuje ścisły podział na strukturę (HTML), stylizację (CSS) i logikę kliencką (JS). Taka architektura zapewnia wysoką czytelność kodu, ułatwia debugowanie oraz pozwala na bezproblemową rozbudowę systemu o nowe funkcjonalności, strony. Główne elementy struktury to:

- **server.js:** To najważniejszy plik serwera. Inicjalizuje aplikację Express, łączy się z bazą danych i importuje wszystkie trasy (routes). To tutaj "startuje" całe zaplecze kina.
- **Grupa routes/:** Tutaj zdefiniowano, co serwer ma zrobić po odebraniu zapytania.
 - **auth.js:** Logowanie i rejestracja.
 - **movies.js:** Pobieranie listy filmów z bazy.
 - **reservations.js:** Logika rezerwowania miejsc (sprawdzanie czy wolne, zapis do bazy).
- **Grupa middleware/ (Strażnicy):** Pliki te kontrolują dostęp. **authMiddleware.js** sprawdza, czy użytkownik jest zalogowany (token JWT), a **adminOnly.js** pilnuje, by tylko administrator mógł np. dodawać filmy.
- **cron.js (Automatyzacja):** Odpowiedzialny za aktualizację repertuaru. **cron.js** wyznacza czas
- **tmdb.js: Skrypt do pobrania filmów z zewnętrznej bazy TMDB**
- **db.js + .env:** Sercem danych jest **db.js** (konfiguracja połączenia z MySQL), natomiast **.env** to bezpieczny schowek na hasła i klucze, do którego kod odwołuje się dynamicznie.

3.2 Wykorzystane Technologie i ich Zastosowanie

3.2.1 Backend

- Node.js & Express.js

- Zastosowanie: Główny silnik aplikacji. Express.js zarządza routingiem (ścieżkami), obsługuje żądania HTTP od klienta i integruje wszystkie moduły backendowe.
- Lokalizacja: backend/server.js (główny plik), backend/routes/ (podział logiki na moduły).

```
1  require('dotenv').config();
2  require('./cron');
3  const express = require('express');
4  const bodyParser = require('body-parser');
5  const cors = require('cors');
6  const path = require('path');
7
8
9  const authRoutes = require('./routes/auth');
10 const userRoutes = require('./routes/users');
11 const movieRoutes = require('./routes/movies');
12 const reservationRoutes = require('./routes/reservations');
13 const adminRoutes = require('./routes/admin_control');
14
15
16 const app = express();
17 const PORT = process.env.PORT;
18
19 app.use(cors());
20 app.use(bodyParser.json());
21 app.use((req, res, next) => {
22   res.set('Cache-Control', 'no-store, no-cache, must-revalidate, private');
23   next();
24 });
25
26
27
28 app.use(express.static(path.join(__dirname, '../frontend/html')));
29 app.use('/css', express.static(path.join(__dirname, '../frontend/css')));
30 app.use('/js', express.static(path.join(__dirname, '../frontend/js')));
31 app.use('/img', express.static(path.join(__dirname, '../frontend/img')));
32
33 app.use('/auth', authRoutes);
34 app.use('/users', userRoutes);
35 app.use('/movies', movieRoutes);
36 app.use('/reservations', reservationRoutes);
37 app.use('/admin_control', adminRoutes);
38
39 app.listen(PORT, () => {
40   console.log(`Serwer działa na http://localhost:\${PORT}`);
41 });
42
```

Rys 5. Plik server.js

Plik odpowiada za konfigurację i uruchomienie serwera backendowego opartego na frameworku Express.js. Na początku wczytywane są zmienne środowiskowe z pliku .env oraz uruchamiany jest moduł realizujący zadania cykliczne. Następnie importowane są niezbędne biblioteki oraz moduły obsługujące poszczególne funkcjonalności aplikacji, takie jak autoryzacja użytkowników, zarządzanie kontami, obsługa filmów, system rezerwacji oraz panel administracyjny. Aplikacja konfiguruje globalne mechanizmy pośredniczące, umożliwiające obsługę żądań JSON, komunikację między frontendem i backendem oraz zwiększenie bezpieczeństwa poprzez wyłączenie buforowania odpowiedzi. Serwer udostępnia również pliki statyczne frontendu. Na końcu rejestrowane są trasy API, a aplikacja zostaje uruchomiona na porcie zdefiniowanym w zmiennych środowiskowych.

- **Poszczególne API:**

W warstwie backendowej zastosowano mechanizm Express Router, który umożliwia modularny podział aplikacji na mniejsze, niezależne komponenty logiczne. Każda grupa funkcjonalna (np. autoryzacja, zarządzanie repertuarem, rezerwacje) została wydzielona do osobnego pliku w katalogu routes/.

```
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const getConnection = require('../db');
const router = express.Router();
const validator = require('validator');
const authenticateToken = require('../middleware/authMiddleware');

const JWT_SECRET = process.env.JWT_SECRET;
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,30}$/;
const usernameRegex = /^(?=.*[a-zA-Z])[a-zA-Z0-9_-]{3,20}$/;

router.post('/register', async (req, res) => {
  let conn;
  try {
    const { uname, email, psw, confirmPassword } = req.body;

    if (!uname || !email || !psw || !confirmPassword) {
      return res.status(400).json({ success: false, message: 'Wszystkie pola są wymagane.' });
    }

    if (uname.startsWith(`etikino_`)) {
      return res.status(400).json({ success: false, message: 'Zakaz tworzenia konta rozpoczynającego się na etikino_' });
    }

    if (psw !== confirmPassword) {
      return res.status(400).json({ success: false, message: 'Hasła nie są identyczne.' });
    }
  }
}
```

```

    if (!usernameRegex.test(uname)) {
      return res.status(400).json({
        success: false,
        message: 'Nazwa użytkownika musi mieć 3-20 znaków, zawierać przynajmniej
jedną literę i może zawierać cyfry, _ lub -.'
      });
    }
    if (!validator.isEmail(email)) {
      return res.status(400).json({ success: false, message: 'Nieprawidłowy adres e-mail.'
});
    }

    if (!passwordRegex.test(psw)) {
      return res.status(400).json({
        success: false,
        message: 'Hasło musi mieć minimum 8, maksymalnie 30 znaków, zawierać małą i
wielką literę, cyfrę oraz znak specjalny.'
      });
    }

    conn = await getConnection();

    const [existing] = await conn.execute(
      'SELECT id,username,email FROM users WHERE LOWER(username)=LOWER(?)
OR LOWER(email)=LOWER(?) LIMIT 1',
      [uname, email]
    );
    if (existing.length > 0 && existing[0].username.toLowerCase() ===
uname.toLowerCase()) {
      return res.status(409).json({ success: false, message: 'Użytkownik o takiej nazwie już
istnieje.' });
    }
    if (existing.length > 0 && existing[0].email.toLowerCase() === email.toLowerCase()) {
      return res.status(409).json({ success: false, message: 'E-mail już zajęty.' });
    }

    const hash = await bcrypt.hash(psw, 10);

    await conn.execute(
      'INSERT INTO users (username, email, password, created_at) VALUES (?, ?, ?,
NOW())',
      [uname, email, hash]
    );

    return res.json({ success: true, message: 'Zarejestrowano pomyślnie\nZaloguj się.' });

```



```

    } catch (err) {
      console.error(err);
      return res.status(500).json({ success: false, message: 'Błąd serwera.' });
    }
  });

router.post('/login', async (req, res) => {
  try {
    const { uname, psw } = req.body;
    if (!uname || !psw) return res.status(400).json({ success: false, message: 'Wszystkie pola są wymagane.' });

    const conn = await getConnection();
    const [rows] = await conn.execute(
      'SELECT id, username, password, role, session_token FROM users WHERE LOWER(username)=LOWER(?)',
      [uname]
    );

    if (rows.length === 0) {
      await conn.end();
      return res.status(401).json({ success: false, message: 'Nieprawidłowy login lub hasło.' });
    }

    const user = rows[0];
    const match = await bcrypt.compare(psw, user.password);
    if (!match) {
      await conn.end();
      return res.status(401).json({ success: false, message: 'Nieprawidłowy login lub hasło.' });
    }

    // Sprawdzamy, czy istnieje aktywny token w bazie
    if (user.session_token) {
      try {
        jwt.verify(user.session_token, JWT_SECRET);
        // Token ważny => odrzuć logowanie
        await conn.end();
        return res.status(403).json({ success: false, message: 'Konto jest już zalogowane na innym urządzeniu.' });
      } catch (err) {
        // Token wygasł => kontynuujemy logowanie
      }
    }
  }
});

```

```

    // Tworzymy nowy token
    const token = jwt.sign({ id: user.id, username: user.username, role: user.role },
    JWT_SECRET, { expiresIn: '1h' });

    // Zapis nowego tokena w bazie
    await conn.execute('UPDATE users SET session_token = ? WHERE id = ?', [token,
    user.id]);
    await conn.end();

    res.json({ success: true, token, role: user.role });

  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.post('/logout', authenticateToken, async (req, res) => {
  try {
    const conn = await getConnection();
    await conn.execute('UPDATE users SET session_token = NULL WHERE id = ?',
    [req.user.id]);
    await conn.end();
    return res.json({ success: true, message: 'Wylogowano pomyślnie.' });
  } catch (err) {
    console.error(err);
    return res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

module.exports = router;

```

Rys 6. Plik auth.js

Plik odpowiada za obsługę procesu rejestracji, logowania oraz wylogowywania użytkowników w aplikacji. Wykorzystuje framework Express do tworzenia tras API, bibliotekę bcrypt do bezpiecznego haszowania haseł oraz jsonwebtoken do generowania tokenów JWT służących do uwierzytelniania sesji użytkowników. Kluczowym elementem bezpieczeństwa jest walidacja danych wejściowych, realizowana przed jakimkolwiek zapytaniem do bazy danych. Wyrażenia regularne (Regex): Zdefiniowane wzorce passwordRegex oraz usernameRegex wymuszają na użytkowniku stosowanie bezpiecznych danych. Walidacja hasła: Regex pilnuje tzw. "polityki silnych haseł" (wymagane minimum 8 znaków, mała i wielka litera, cyfra oraz znak specjalny). Walidacja loginu: Wyrażenie regularne dba o to, by nazwa użytkownika miała odpowiednią długość i nie zawierała niedozwolonych znaków, co zapobiega błędom w wyświetlaniu interfejsu. Biblioteka Validator została wykorzystana do specjalistycznej weryfikacji adresu e-mail (validator.isEmail). Dzięki niej system ma

pewność, że wprowadzony ciąg znaków jest faktycznie poprawnym formatem poczty elektronicznej. Podczas rejestracji system sprawdza również unikalność loginu i maila oraz zakazuje tworzenia kont z zastrzeżonym prefiksem etikino_ (zarezerwowanym dla kont systemowych/admina). Dopiero po pomyślnym przejściu wszystkich testów, hasło jest haszowane i zapisywane w bazie MySQL. Proces logowania sprawdza poprawność podanych danych i porównuje hasło z hashem w bazie danych. Jeśli użytkownik posiada już aktywny token, nowe logowanie jest blokowane do czasu wygaśnięcia poprzedniego tokenu. W przypadku poprawnego logowania generowany jest nowy token JWT, który jest przechowywany w bazie i przekazywany użytkownikowi w odpowiedzi. Funkcja wylogowania usuwa token sesji z bazy danych, unieważniając w ten sposób sesję użytkownika. Wszystkie operacje bazodanowe wykonywane są asynchronicznie przy użyciu połączenia z bazą MySQL.

```
const express = require('express');
const getConnection = require('../db');
const bcrypt = require('bcrypt');
const validator = require('validator');
const authenticateToken = require('../middleware/authMiddleware');
const router = express.Router();

router.get('/account', authenticateToken, async (req, res) => {
  try {
    const conn = await getConnection();
    const [rows] = await conn.execute('SELECT username,email,role FROM users
WHERE id = ?', [req.user.id]);
    await conn.end();

    if (rows.length === 0) return res.status(404).json({ success: false, message:
'Użytkownik nie istnieje.' });
    res.json({ success: true, user: rows[0] });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,30}$/;
const usernameRegex = /^(?=.*[a-zA-Z])[a-zA-Z0-9_-]{3,20}$/;

router.put('/update', authenticateToken, async (req, res) => {
  const {
    username,
    email,
    currentPassword,
    newPassword,
    confirmPassword
  } = req.body;
```

```
const id = req.user.id;

if (!currentPassword) {
  return res.status(400).json({
    success: false,
    message: "Musisz podać aktualne hasło, aby wykonać zmianę."
  });
}

try {
  const conn = await getConnection();

  // Pobierz aktualne dane użytkownika
  const [userRows] = await conn.execute(
    "SELECT username, email, password FROM users WHERE id = ? LIMIT 1",
    [id]
  );

  if (userRows.length === 0) {
    return res.status(404).json({ success: false, message: "Użytkownik nie istnieje." });
  }

  const user = userRows[0];

  const isPasswordValid = await bcrypt.compare(currentPassword, user.password);
  if (!isPasswordValid) {
    return res.status(401).json({
      success: false,
      message: "Błędne aktualne hasło."
    });
  }

  const updates = [];
  const values = [];

  if (username !== undefined) {
    if (username === user.username) {
      return res.status(400).json({
        success: false,
        message: "Nowa nazwa użytkownika nie może być taka sama jak obecna."
      });
    }
  }
}
```

```

if (!usernameRegex.test(username)) {
  return res.status(400).json({
    success: false,
    message:
      'Nazwa użytkownika musi mieć 3–20 znaków, zawierać litery i może też _ lub -.'
  });
}

const [existsUser] = await conn.execute(
  "SELECT id FROM users WHERE LOWER(username)=LOWER(?) AND id != ?",
  [username, id]
);

if (existsUser.length > 0) {
  return res.status(409).json({
    success: false,
    message: "Ta nazwa użytkownika jest już zajęta."
  });
}

updates.push("username = ?");
values.push(username);
}

if (email !== undefined) {
  if (email === user.email) {
    return res.status(400).json({
      success: false,
      message: "Nowy email nie może być taki sam jak obecny."
    });
  }

  if (!validator.isEmail(email)) {
    return res.status(400).json({
      success: false,
      message: "Nieprawidłowy email."
    });
  }

  const [existsEmail] = await conn.execute(
    "SELECT id FROM users WHERE LOWER(email)=LOWER(?) AND id != ?",
    [email, id]
  );

```

```
if (existsEmail.length > 0) {
  return res.status(409).json({
    success: false,
    message: "Ten email jest już używany."
  });
}

updates.push("email = ?");
values.push(email);
}

if (newPassword !== undefined || confirmPassword !== undefined) {

  if (!newPassword || !confirmPassword) {
    return res.status(400).json({
      success: false,
      message: "Podaj nowe hasło i powtórz nowe hasło."
    });
  }

  if (newPassword === currentPassword) {
    return res.status(400).json({
      success: false,
      message: "Nowe hasło nie może być takie samo jak obecne."
    });
  }

  if (newPassword !== confirmPassword) {
    return res.status(400).json({
      success: false,
      message: "Nowe hasła nie są identyczne."
    });
  }

  if (!passwordRegex.test(newPassword)) {
    return res.status(400).json({
      success: false,
      message:
        "Hasło musi mieć 8–30 znaków, zawierać małą i dużą literę, cyfrę i znak specjalny."
    });
  }

  const hash = await bcrypt.hash(newPassword, 10);
```

```

    updates.push("password = ?");
    values.push(hash);
  }

  if (updates.length === 0) {
    return res.status(400).json({
      success: false,
      message: "Nie podano żadnych danych do zmiany."
    });
  }

  const sql = `UPDATE users SET ${updates.join(", ")} WHERE id = ?`;
  values.push(id);

  await conn.execute(sql, values);
  await conn.end();

  res.json({
    success: true,
    forceLogout: true,
    message: "Dane zostały zmienione. Zaloguj się ponownie."
  });

} catch (err) {
  console.error(err);
  res.status(500).json({
    success: false,
    message: "Błąd serwera."
  });
}
});

module.exports = router;

```

Rys 7. Plik users.js

Plik odpowiada za zarządzanie kontem użytkownika w aplikacji, umożliwiając pobieranie danych oraz aktualizację informacji takich jak nazwa użytkownika, e-mail czy hasło. Wykorzystuje framework Express do tworzenia tras API, bibliotekę bcrypt do bezpiecznego porównywania i haszowania haseł oraz validator do walidacji adresów e-mail. Wszystkie operacje są chronione tokenem JWT, dzięki czemu tylko zalogowany użytkownik może odczytać lub zmienić swoje dane. Trasa GET /account

zwraca podstawowe informacje o użytkowniku, takie jak nazwa, e-mail i rola, pobierane z bazy danych MySQL. Trasa PUT /update umożliwia aktualizację danych użytkownika przy zachowaniu szeregu zabezpieczeń: weryfikowane jest aktualne hasło, sprawdzana jest poprawność i unikalność nowej nazwy i e-maila, a nowe hasło musi spełniać określone wymagania bezpieczeństwa. W przypadku zmiany danych użytkownik jest proszony o ponowne zalogowanie się, aby zapewnić bezpieczeństwo sesji. Wszystkie operacje bazodanowe wykonywane są asynchronicznie przy użyciu połączenia z bazą MySQL.

```
const express = require('express');
const getConnection = require('../db');
const authenticateToken = require('../middleware/authMiddleware');
const router = express.Router();

router.get('/all_showings', async (req, res) => {
  try {
    const conn = await getConnection();

    const [rows] = await conn.execute(`
      SELECT
        m.id AS movie_id,
        m.title,
        m.poster,
        m.description,
        m.genre,
        m.release_date,
        o.id AS on_screen_id,
        o.start_time,
        o.price,
        s.name AS screen_name
      FROM movies m
      JOIN on_screen o ON m.id = o.movie_id
      JOIN screens s ON o.screen_id = s.id
      WHERE o.start_time > NOW()
      ORDER BY m.id, o.start_time
    `);

    await conn.end();
    res.json({ success: true, showings: rows });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.get('/movies_all', async (req, res) => {
  try {
```



```

const conn = await getConnection();
const [rows] = await conn.execute('SELECT * FROM `movies`');
await conn.end();
res.json({ success: true, movies_all: rows });
} catch (err) {
res.status(500).json({ success: false, message: 'Błąd serwera.' });
}
});

router.get('/movies_playing', async (req, res) => {
  try {
    const conn = await getConnection();
    const [rows] = await conn.execute('SELECT m.id, m.title, m.poster, m.description,
m.genre, m.release_date,o.id AS `o_id`, o.start_time, o.price, s.name FROM movies m
JOIN on_screen o ON m.id = o.movie_id JOIN screens s ON o.screen_id = s.id WHERE
o.start_time = ( SELECT MIN(o2.start_time) FROM on_screen o2 WHERE o2.movie_id
= m.id AND o2.start_time > NOW() );');
    await conn.end();
    res.json({ success: true, movies_playing: rows });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.get('/movie/:id', async (req, res) => {
  const id = req.params.id;
  try {
    const conn = await getConnection();
    const [rows] = await conn.execute('SELECT * FROM `movies` WHERE id = ?', [id]);
    await conn.end();

    if (!rows || rows.length === 0) {
      return res.status(404).json({ success: false, message: 'Film nie znaleziony.' });
    }

    res.json({ success: true, movie: rows[0] });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.get('/movie/:id/showings', async (req, res) => {
  const id = req.params.id;
  try {
    const conn = await getConnection();

```

```

const [rows] = await conn.execute(
  `SELECT on_screen.start_time, on_screen.price, screens.name, on_screen.id
  FROM on_screen
  JOIN screens ON on_screen.screen_id = screens.id
  WHERE on_screen.movie_id = ? AND start_time > NOW()`, [id]
);
await conn.end();
res.json({ success: true, showings: rows });
} catch (err) {
  console.error(err);
  res.status(500).json({ success: false, message: 'Błąd serwera.' });
}
});

router.get('/movie/:id/comments', async (req, res) => {
  const conn = await getConnection();
  const movieId = req.params.id;
  try {
    const [rows] = await conn.execute(
      'SELECT c.id, c.user_id, c.content, c.created_at, u.username FROM comments c
      JOIN users u ON c.user_id = u.id WHERE c.movie_id = ? ORDER BY c.created_at
      DESC',
      [movieId]
    );
    res.json({ success: true, comments: rows });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.post('/movie/:id/comments_add', authenticateToken, async (req, res) => {
  const conn = await getConnection();
  const movieId = req.params.id;
  const userId = req.user.id;
  const { text } = req.body;

  if (!text || text.trim() === '') {
    return res.status(400).json({ success: false, message: 'Komentarz nie może być pusty.' });
  }

  try {
    await conn.execute(
      `INSERT INTO comments (movie_id, user_id, content) VALUES (?, ?, ?)`,
      [movieId, userId, text]
    );
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

```

```

);
res.json({ success: true, message: 'Komentarz dodany.' });
} catch (err) {
  console.error('Błąd dodawania komentarza:', err);
  res.status(500).json({ success: false, message: 'Błąd serwera' });
}
});

module.exports = router;

```

Rys 8. Plik *movies.js*

Plik odpowiada za obsługę tras związanych z filmami w aplikacji, w tym wyświetlanie listy filmów, szczegółów wybranego filmu, repertuaru seansów oraz komentarzy użytkowników. Wykorzystuje framework Express do tworzenia tras API oraz moduł `authenticateToken`, aby chronić operacje wymagające zalogowanego użytkownika. Wszystkie zapytania do bazy danych MySQL wykonywane są asynchronicznie przy użyciu funkcji `getConnection`. Trasy GET umożliwiają pobranie pełnej listy filmów, aktualnie wyświetlanych filmów, szczegółów pojedynczego filmu oraz jego seansów i komentarzy. Trasa POST `/movie/:id/comments_add` pozwala zalogowanemu użytkownikowi dodać komentarz do wybranego filmu, przy czym treść komentarza jest weryfikowana pod kątem pustych wartości. W przypadku błędów serwera lub braku danych, odpowiednie komunikaty zwracane są w formacie JSON z odpowiednim kodem HTTP.

```

const express = require('express');
const getConnection = require('../db');
const authenticateToken = require('../middleware/authMiddleware');
const router = express.Router();
async function updateEndedReservations(conn) {
  await conn.execute(`
    UPDATE reservations r
    JOIN on_screen os ON r.on_screen_id = os.id
    SET r.status = 'ended'
    WHERE os.start_time < NOW() AND r.status != 'ended'
  `);
}
router.get("/seats/:onScreenId", async (req, res) => {
  const id = req.params.onScreenId;

  try {
    const conn = await getConnection();

    const [seats] = await conn.execute(`
      SELECT ss.id, ss.seat_row, ss.seat_number,
      CASE WHEN r.id IS NOT NULL THEN 1 ELSE 0 END AS taken
      FROM screen_seats ss
    `);
  } catch (err) {
    console.error('Błąd pobierania miejsc:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

```

```

        JOIN on_screen os ON os.screen_id = ss.screen_id
        LEFT JOIN reservations r
            ON r.seat_id = ss.id AND r.on_screen_id = os.id
        WHERE os.id = ?
        ORDER BY ss.seat_row, ss.seat_number
    `, [id]);

    res.json({ success: true, seats });
    console.log("Fetching seats for onScreenId:", id);
} catch (err) {
    console.error(err);
    res.status(500).json({ success: false });
}
});

router.post("/reserve", authenticateToken, async (req, res) => {
    const userId = req.user.id; // teraz będzie dostępne
    const { onScreenId, seats } = req.body;

    if (!Array.isArray(seats) || seats.length === 0) {
        return res.status(400).json({ success: false, msg: "Nie wybrano miejsc" });
    }

    try {
        const conn = await getConnection();

        const placeholders = seats.map(() => '?').join(',');
        const [takenSeats] = await conn.execute(`
            SELECT seat_id FROM reservations
            WHERE on_screen_id = ? AND seat_id IN (${placeholders})
        `, [onScreenId, ...seats]);

        if (takenSeats.length > 0) {
            return res.status(400).json({ success: false, msg: "Niektóre miejsca są już zajęte" });
        }

        const values = seats.map(seatId => [userId, onScreenId, seatId]);
        await conn.query(`
            INSERT INTO reservations (user_id, on_screen_id, seat_id) VALUES ?
        `, [values]);

        res.json({ success: true });
    }
});

```

```

    } catch (err) {
      console.error(err);
      res.status(500).json({ success: false, msg: "Błąd serwera" });
    }
  });

```

// Pobranie rezerwacji dla zalogowanego użytkownika

```

router.get('/my', authenticateToken, async (req, res) => {
  const userId = req.user.id;

```

```

    try {
      const conn = await getConnection();
      await updateEndedReservations(conn);
const [rows] = await conn.execute(`
SELECT
  r.id,
  r.seat_id,
  s.seat_row,
  s.seat_number,
  scr.name AS screen_name,
  m.title,
  os.start_time,
  r.status
FROM reservations r
JOIN screen_seats s ON r.seat_id = s.id
JOIN on_screen os ON r.on_screen_id = os.id
JOIN movies m ON os.movie_id = m.id
JOIN screens scr ON os.screen_id = scr.id
WHERE r.user_id = ?
ORDER BY m.title, os.start_time, s.seat_row, s.seat_number
`, [userId]);
      await conn.end();

```

```

      res.json({ success: true, reservations: rows });
    } catch (err) {
      console.error(err);
      res.status(500).json({ success: false });
    }
  });

```

```

router.delete('/cancel/:id', authenticateToken, async (req, res) => {
  const reservationId = req.params.id;
  const userId = req.user.id;

```

```

    try {
      const conn = await getConnection();

```

```

const [rows] = await conn.execute(
  'SELECT * FROM reservations WHERE id = ? AND user_id = ? AND status =
"reserved"',
  [reservationId, userId]
);

if (!rows.length) {
  await conn.end();
  return res.status(400).json({ success: false, message: 'Nie można anulować tej
rezerwacji.' });
}

await conn.execute('DELETE FROM reservations WHERE id = ?', [reservationId]);
await conn.end();

res.json({ success: true, message: 'Rezerwacja anulowana.' });
} catch (err) {
  console.error(err);
  res.status(500).json({ success: false, message: 'Błąd serwera.' });
}
});

router.delete('/delete/:id', authenticateToken, async (req, res) => {
  const reservationId = req.params.id;
  const userId = req.user.id;

  try {
    const conn = await getConnection();

    const [rows] = await conn.execute(
      'SELECT * FROM reservations WHERE id = ? AND user_id = ? AND status =
"ended"',
      [reservationId, userId]
    );

    if (!rows.length) {
      await conn.end();
      return res.status(400).json({ success: false, message: 'Nie można anulować tej
rezerwacji.' });
    }

    await conn.execute('DELETE FROM reservations WHERE id = ?', [reservationId]);
    await conn.end();
  }
});

```

```

    res.json({ success: true, message: 'Seans usunięty' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});

router.post('/pay/:id', authenticateToken, async (req, res) => {
  const reservationId = req.params.id;
  const userId = req.user.id;

  try {
    const conn = await getConnection();

    const [rows] = await conn.execute(
      'SELECT * FROM reservations WHERE id = ? AND user_id = ? AND status = "reserved"',
      [reservationId, userId]
    );

    if (!rows.length) {
      await conn.end();
      return res.status(400).json({ success: false, message: 'Nie można opłacić tej rezerwacji.' });
    }

    await conn.execute('UPDATE reservations SET status = "paid" WHERE id = ?', [reservationId]);
    await conn.end();

    res.json({ success: true, message: 'Rezerwacja opłacona.' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera.' });
  }
});
module.exports = router;

```

Rys 9. Plik reservations.js

Plik odpowiada za zarządzanie rezerwacjami miejsc w kinie. Wykorzystuje framework Express do tworzenia tras API oraz moduł authenticateToken do ochrony operacji wymagających zalogowanego użytkownika. Wszystkie zapytania do bazy danych MySQL wykonywane są asynchronicznie przy użyciu funkcji getConnection. Trasy GET umożliwiają pobranie dostępnych miejsc dla danego seansu oraz przeglądanie

własnych rezerwacji wraz ze szczegółami filmu, sali i statusu rezerwacji. Trasa POST /reserve pozwala zalogowanemu użytkownikowi dokonać rezerwacji wybranych miejsc, przy czym najpierw sprawdzany jest ich aktualny status w bazie. Dodatkowo dostępne są trasy do anulowania (/cancel/:id) lub usunięcia zakończonych rezerwacji (/delete/:id) oraz do opłacenia rezerwacji (/pay/:id). Funkcja updateEndedReservations automatycznie zmienia status rezerwacji na „ended” dla seansów, które już się odbyły, co zapewnia spójność danych. W przypadku błędów serwera lub niepoprawnych operacji, użytkownik otrzymuje odpowiednie komunikaty w formacie JSON.

```
const express = require('express');
const getConnection = require('../db');
const authenticateToken = require('../middleware/authMiddleware');
const adminOnly = require('../middleware/adminOnly');
const router = express.Router();

router.get('/movies', authenticateToken, adminOnly, async (req, res) => {
  try {
    const conn = await getConnection();
    const [movies] = await conn.execute('SELECT * FROM movies');
    await conn.end();
    res.json(movies);
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Pobranie sal
router.get('/screens', authenticateToken, adminOnly, async (req, res) => {
  try {
    const conn = await getConnection();
    const [screens] = await conn.execute('SELECT * FROM screens');
    await conn.end();
    res.json(screens);
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Pobranie wyświetlanych filmów
router.get('/on_screen', authenticateToken, adminOnly, async (req, res) => {
  try {
    const conn = await getConnection();
    const [onScreen] = await conn.execute(`
      SELECT os.id, os.movie_id, os.screen_id, os.start_time, os.price,
```



```

        m.title, m.poster, s.name AS screen_name
    FROM on_screen os
    JOIN movies m ON os.movie_id = m.id
    JOIN screens s ON os.screen_id = s.id WHERE os.status = 'upcoming';
`);
    await conn.end();
    res.json(onScreen);
} catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
}
});

router.post('/on_screen', authenticateToken, adminOnly, async (req, res) => {
    const { movie_id, screen_id, start_time, price } = req.body;
    if (!movie_id || !screen_id || !start_time || !price) {
        return res.status(400).json({ success: false, message: 'Brak wymaganych danych'
    });
    }

    try {
        const conn = await getConnection();

        // Pobierz czas trwania filmu w minutach
        const [movieRows] = await conn.execute(
            'SELECT duration_minutes FROM movies WHERE id = ?',
            [movie_id]
        );
        if (movieRows.length === 0) {
            await conn.end();
            return res.status(404).json({ success: false, message: 'Film nie istnieje' });
        }
        const durationMinutes = movieRows[0].duration_minutes;
        const newStart = new Date(start_time);
        const newEnd = new Date(newStart.getTime() + durationMinutes * 60 * 1000);

        // Pobranie wszystkich seansów w tej samej sali
        const [existing] = await conn.execute(
            'SELECT id, start_time, movie_id FROM on_screen WHERE screen_id = ? AND
status = ?',
            [screen_id, 'upcoming']
        );

        for (let show of existing) {
            // Pobierz czas trwania istniejącego filmu

```

```

const [existingMovie] = await conn.execute(
  'SELECT duration_minutes FROM movies WHERE id = ?',
  [show.movie_id]
);
const existingDuration = existingMovie[0].duration_minutes;
const existingStart = new Date(show.start_time);
const existingEnd = new Date(existingStart.getTime() + existingDuration * 60 *
1000);

if (newStart < existingEnd && newEnd > existingStart) {
  await conn.end();
  return res.status(400).json({
    success: false,
    message: 'Wybrana sala jest już zajęta w tym czasie.'
  });
}

// Dodajemy seans
await conn.execute(
  'INSERT INTO on_screen (movie_id, screen_id, start_time, price) VALUES (?, ?,
?, ?)',
  [movie_id, screen_id, start_time, price]
);

await conn.end();
res.json({ success: true });

} catch (err) {
  console.error(err);
  res.status(500).json({ success: false, message: 'Błąd serwera' });
}
});

router.delete('/on_screen/:id', authenticateToken, adminOnly, async (req, res) => {
  const id = req.params.id;
  try {
    const conn = await getConnection();
    await conn.execute('DELETE FROM on_screen WHERE id = ?', [id]);
    await conn.end();
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
}

```

```

});

// Pobierz wszystkich użytkowników (oprócz adminów)
router.get('/users/all', authenticateToken, adminOnly, async (req, res) => {
  let conn;
  try {
    conn = await getConnection();
    const [users] = await conn.execute(
      'SELECT id, username, email FROM users WHERE role != ? ORDER BY id
ASC',
      ['admin']
    );
    await conn.end();
    res.json({ success: true, users });
  } catch (err) {
    if (conn) await conn.end();
    console.error(err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Szczegóły użytkownika wraz z komentarzami i rezerwacjami
router.get('/users/:id/full-details', authenticateToken, adminOnly, async (req, res) => {
  const userId = req.params.id;
  let conn;

  try {
    conn = await getConnection();

    const [userRows] = await conn.execute(
      'SELECT id, username, email, role FROM users WHERE id = ?',
      [userId]
    );

    if (userRows.length === 0) {
      await conn.end();
      return res.status(404).json({ success: false, message: 'Użytkownik nie istnieje' });
    }

    const [comments] = await conn.execute(`
      SELECT c.id, c.content, c.created_at, m.title AS movie_title
      FROM comments c
      LEFT JOIN movies m ON c.movie_id = m.id
      WHERE c.user_id = ?
      ORDER BY c.created_at DESC
    `);
  }
});

```

```

    `, [userId]);

    const [reservations] = await conn.execute(`
      SELECT r.id, r.status, m.title AS movie_title, os.start_time,
             s.name AS screen_name, ss.seat_row, ss.seat_number
      FROM reservations r
      LEFT JOIN on_screen os ON r.on_screen_id = os.id
      LEFT JOIN movies m ON os.movie_id = m.id
      LEFT JOIN screens s ON os.screen_id = s.id
      LEFT JOIN screen_seats ss ON r.seat_id = ss.id
      WHERE r.user_id = ?
      ORDER BY r.id DESC
    `, [userId]);

    await conn.end();

    res.json({ success: true, user: userRows[0], comments, reservations });

  } catch (err) {
    if (conn) await conn.end();
    console.error('FULL DETAILS ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Aktualizacja danych użytkownika
router.put('/users/:id/update', authenticateToken, adminOnly, async (req, res) => {
  const { username, email, password } = req.body;
  const userId = req.params.id;
  let conn;

  try {
    conn = await getConnection();

    if (username) {
      await conn.execute('UPDATE users SET username = ? WHERE id = ?',
[username, userId]);
    }
    if (email) {
      await conn.execute('UPDATE users SET email = ? WHERE id = ?', [email,
userId]);
    }
    if (password) {
      const hashed = await bcrypt.hash(password, 10);
      await conn.execute('UPDATE users SET password = ? WHERE id = ?', [hashed,
userId]);
    }
  }
});

```

```

    }

    await conn.end();
    res.json({ success: true, message: 'Dane użytkownika zaktualizowane' });

  } catch (err) {
    if (conn) await conn.end();
    console.error('UPDATE USER ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Usuń użytkownika
router.delete('/users/:id', authenticateToken, adminOnly, async (req, res) => {
  const userId = req.params.id;
  let conn;

  try {
    conn = await getConnection();
    const [result] = await conn.execute('DELETE FROM users WHERE id = ?', [userId]);
    await conn.end();

    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'Użytkownik nie istnieje' });
    }

    res.json({ success: true, message: 'Użytkownik usunięty' });
  } catch (err) {
    if (conn) await conn.end();
    console.error('DELETE USER ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

// Usuń komentarz
router.delete('/comments/:id', authenticateToken, adminOnly, async (req, res) => {
  const commentId = req.params.id;
  let conn;

  try {
    conn = await getConnection();
    const [result] = await conn.execute('DELETE FROM comments WHERE id = ?',
[commentId]);
    await conn.end();

```

```

    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'Komentarz nie istnieje' });
    }

    res.json({ success: true, message: 'Komentarz usunięty' });
  } catch (err) {
    if (conn) await conn.end();
    console.error('DELETE COMMENT ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

```

// Usuń rezerwację

```

router.delete('/reservations/:id', authenticateToken, adminOnly, async (req, res) => {
  const id = req.params.id;
  let conn;

  try {
    conn = await getConnection();
    const [result] = await conn.execute('DELETE FROM reservations WHERE id = ?',
[id]);
    await conn.end();

    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'Rezerwacja nie istnieje' });
    }

    res.json({ success: true, message: 'Rezerwacja usunięta' });
  } catch (err) {
    if (conn) await conn.end();
    console.error('DELETE RESERVATION ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

```

// Opłać rezerwację (zmiana statusu na "paid")

```

router.post('/reservations/:id/pay', authenticateToken, adminOnly, async (req, res) => {
  const id = req.params.id;
  let conn;

  try {
    conn = await getConnection();
    const [result] = await conn.execute(
      'UPDATE reservations SET status = ? WHERE id = ?',
      ['paid', id]
    );
    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'Rezerwacja nie istnieje' });
    }
    res.json({ success: true, message: 'Rezerwacja opłacony' });
  } catch (err) {
    if (conn) await conn.end();
    console.error('UPDATE RESERVATION ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
  }
});

```

```

    );
    await conn.end();

    if (result.affectedRows === 0) {
        return res.status(404).json({ success: false, message: 'Rezerwacja nie istnieje' });
    }

    res.json({ success: true, message: 'Rezerwacja opłacona' });
} catch (err) {
    if (conn) await conn.end();
    console.error('PAY RESERVATION ERROR:', err);
    res.status(500).json({ success: false, message: 'Błąd serwera' });
}
});

```

module.exports = router;

Rys 10. Plik admin_control.js

Plik odpowiada za funkcje administracyjne w aplikacji, umożliwiając zarządzanie filmami, salami, seansami oraz użytkownikami. Wszystkie trasy są chronione tokenem JWT oraz dodatkowo sprawdzają, czy zalogowany użytkownik ma uprawnienia administratora (adminOnly). Operacje na bazie danych MySQL wykonywane są asynchronicznie przy użyciu funkcji getConnection. Administrator może przeglądać wszystkie filmy, sale i nadchodzące seanse, dodawać nowe seanse oraz usuwać istniejące. Ponadto możliwe jest zarządzanie użytkownikami: przeglądanie pełnych szczegółów, aktualizacja danych, usuwanie kont, a także usuwanie komentarzy i rezerwacji. Dodatkowo administrator może zmieniać status rezerwacji na „paid”. W przypadku błędów serwera lub braku danych, aplikacja zwraca odpowiednie komunikaty w formacie JSON z kodami HTTP.

- **MySQL**

- **Zastosowanie:** Relacyjna baza danych przechowująca tabele: użytkowników, filmów, seansów oraz rezerwacji.
- **Lokalizacja:** backend/db.js (konfiguracja połączenia), pliki .sql

```

const mysql = require('mysql2/promise');

const HOST = process.env.HOST;
const USER = process.env.USER;
const PASSWORD = process.env.PASSWORD;
const DATABASE = process.env.DATABASE;

const dbConfig = {
    host: HOST,
    user: USER,
    password: PASSWORD,
    database: DATABASE

```

```
};

async function getConnection() {
  return mysql.createConnection(dbConfig);
}

module.exports = getConnection;
```

Rys 11. Plik db.js

Plik odpowiada za konfigurację połączenia z bazą danych MySQL. Wykorzystuje bibliotekę mysql2 w trybie asynchronicznym (promise), co umożliwia obsługę zapytań przy użyciu mechanizmu async/await. Dane konfiguracyjne połączenia, takie jak adres serwera, nazwa użytkownika, hasło oraz nazwa bazy danych, pobierane są ze zmiennych środowiskowych, co zwiększa bezpieczeństwo aplikacji. Zdefiniowana funkcja getConnection tworzy i zwraca nowe połączenie z bazą danych na podstawie podanej konfiguracji. Funkcja ta jest eksportowana i wykorzystywana w innych częściach aplikacji do wykonywania operacji na bazie danych.

- **JSON Web Token (JWT)**

- **Zastosowanie:** Bezpieczna autoryzacja użytkowników. Po zalogowaniu serwer generuje token, który pozwala na identyfikację klienta przy rezerwacji biletów bez konieczności ponownego podawania hasła.
- **Lokalizacja:** backend/.env (klucz JWT_SECRET),
backend/middleware/authMiddleware.js.

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;
const getConnection = require('./db');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.status(401).json({ success: false, message: 'Brak tokenu.' });

  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ success: false, message: 'Nieprawidłowy token.' });
    }

    req.user = user;
    next();
  });
}

module.exports = authenticateToken;
```

Rys 12. Plik authMiddleware.js


```
function adminOnly(req, res, next) {
  if (!req.user || !req.user.username) {
    return res.status(401).json({ success: false, message: 'Brak uprawnień.' });
  }
  if (!req.user.role==='admin') {
    return res.status(403).json({ success: false, message: 'Dostęp tylko dla admina.' });
  }

  next();
}

module.exports = adminOnly;
```

Rys 13. Plik adminOnly.js

Pierwszy plik odpowiada za autoryzację użytkowników przy użyciu tokenów JWT. Funkcja authenticateToken sprawdza, czy w nagłówkach żądania znajduje się token, a następnie weryfikuje jego poprawność przy użyciu sekretu przechowywanego w zmiennej środowiskowej. W przypadku braku tokenu lub niepoprawnego tokenu, funkcja zwraca odpowiedni komunikat błędu i kod HTTP. Jeśli token jest poprawny, informacje o użytkowniku są dodawane do obiektu req.user i przekazywane dalej do kolejnych funkcji obsługi żądań. Drugi plik odpowiada za ograniczenie dostępu do funkcji dostępnych tylko dla administratora. Funkcja adminOnly sprawdza, czy zalogowany użytkownik posiada rolę admin. Jeśli użytkownik nie ma odpowiednich uprawnień, serwer zwraca komunikat o braku dostępu z odpowiednim kodem HTTP. Funkcja pozwala kontynuować wykonywanie żądania tylko wtedy, gdy użytkownik jest administratorem.

- **Axios**

- **Zastosowanie:** Biblioteka służąca do wykonywania asynchronicznych zapytań HTTP. W projekcie wykorzystywana w pliku tmdb.js do pobierania danych o filmach z zewnętrznego serwera TMDB.
- **Lokalizacja:** backend/tmdb.js

```
• const fs = require("fs");
• const axios = require("axios");
•
• const TMDB_API_KEY = "4c71aa364bf102c52698d8df1774678a";
• const TOTAL_PAGES = 50;
•
• function escapeSQL(str) {
•   if (!str) return "";
•   return str.replace(/'/g, "''");
• }
•
• async function fetchDiscover(page) {
•   const url =
•     `https://api.themoviedb.org/3/discover/movie?` +
```

```

    `api_key=${TMDB_API_KEY}&language=pl-
    PL&sort_by=popularity.desc&page=${page}`;

    const { data } = await axios.get(url);
    return data.results;
  }

  async function fetchMovieDetails(id) {
    const url =
      `https://api.themoviedb.org/3/movie/${id}?` +
      `api_key=${TMDB_API_KEY}&language=en-
      US&append_to_response=release_dates`; // gatunki po angielsku

    const { data } = await axios.get(url);
    return data;
  }

  function extractAgeRating(details) {
    try {
      const certs = details.release_dates.results;
      const pl = certs.find(c => c.iso_3166_1 === "PL");
      if (!pl || pl.release_dates.length === 0) return "0+";

      return pl.release_dates[0].certification || "0+";
    } catch {
      return "0+";
    }
  }

  //gatunki rozdzielone SLASHEM
  function extractGenres(details) {
    if (!details.genres) return null;
    return details.genres.map(g => g.name).join("/");
  }

  async function delay(ms) {
    return new Promise(r => setTimeout(r, ms));
  }

  async function generateSQL() {
    const movieMap = new Map();

    console.log("Pobieranie filmów...");

    for (let page = 1; page <= TOTAL_PAGES; page++) {
      const movies = await fetchDiscover(page);

```

```

• console.log(`Strona ${page} — ${movies.length} filmów`);
• movies.forEach(m => movieMap.set(m.id, m));
• }
•
• console.log(`Łącznie filmów: ${movieMap.size}`);
•
• let values = [];
•
• for (const movie of movieMap.values()) {
•   const details = await fetchMovieDetails(movie.id);
•
•   const title = escapeSQL(movie.title);
•   const poster = movie.poster_path || "";
•   const description = escapeSQL(details.overview || "");
•   const duration = details.runtime || 0;
•   const ageRating = extractAgeRating(details);
•   const releaseDate = movie.release_date || null;
•   const genre = escapeSQL(extractGenres(details));
•
•   values.push(
•     `('${title}', 'https://image.tmdb.org/t/p/original${poster}', '${description}',
•     ${duration}, ` +
•     ` '${ageRating}', ` +
•     ` '${releaseDate ? `${releaseDate}` : "NULL"}', ` +
•     ` '${genre ? `${genre}` : "NULL"}'`
•   );
•
•   await delay(200);
• }
•
• const sql =
• `INSERT INTO movies
• (title, poster, description, duration_minutes, age_rating, release_date, genre)
• VALUES
• ${values.join(",\n")}`
• `;
•
• fs.writeFileSync("movies_insert.sql", sql, "utf8");
•
• console.log("Zapisano movies_insert.sql");
• }
•
• generateSQL();

```

Rys 14. Plik tmdb.js

Skrypt odpowiada za pobieranie danych o filmach z serwisu TMDb i generowanie gotowego pliku SQL do wstawienia ich do bazy danych. Wykorzystuje bibliotekę axios

do wykonywania zapytań HTTP oraz moduł fs do zapisu danych do pliku. Skrypt pobiera filmy stronami (TOTAL_PAGES = 50) posortowane według popularności, a następnie dla każdego filmu pobiera szczegółowe informacje, takie jak opis, czas trwania, gatunki czy wiekowa klasyfikacja. Dane są odpowiednio przetwarzane i oczyszczane pod kątem SQL, np. znaki ' w tekstach są podwajane, aby uniknąć błędów składniowych. Gatunki filmów są łączone w jeden ciąg znaków oddzielony ukośnikami, a wiekowa klasyfikacja pobierana jest z polskich wydań filmów. Skrypt dodaje krótką przerwę (delay) między kolejnymi żądaniami, aby nie przekroczyć limitów API. Na końcu generowany jest plik movies_insert.sql, zawierający komendę INSERT INTO movies z wartościami wszystkich pobranych filmów, gotowy do użycia w bazie danych.

- **Node-Cron**

- **Zastosowanie:** Harmonogram zadań ("budzik" systemowy). Służy do automatycznego wyzwalania skryptu aktualizacji repertuaru o określonych godzinach, co eliminuje potrzebę ręcznego odświeżania bazy przez administratora.
- **Lokalizacja:** backend/cron.js.

```
• const cron = require('node-cron');
• const getConnection = require('./db');
•
• async function updateEndedShows() {
•   try {
•     const conn = await getConnection();
•     await conn.execute(`
•       UPDATE on_screen
•       SET status = 'ended'
•       WHERE start_time < NOW() AND status != 'ended'
•     `);
•     await conn.end();
•     console.log('Zaktualizowano status zakończonych seansów');
•   } catch (err) {
•     console.error('Błąd przy aktualizacji seansów:', err);
•   }
• }
•
• updateEndedShows();
•
• // Harmonogram co minutę
• cron.schedule('* * * * *', updateEndedShows);
```

Rys 15. Plik cron.js

Plik odpowiada za realizację zadania cyklicznego aktualizującego status seansów filmowych w bazie danych. Wykorzystuje bibliotekę node-cron do harmonogramowania wykonywania zadań oraz moduł obsługujący połączenie z bazą danych. Zdefiniowana funkcja asynchroniczna nawiązuje połączenie z bazą i wykonuje zapytanie SQL, które zmienia status seansów na „ended” w przypadku, gdy ich czas rozpoczęcia jest wcześniejszy od aktualnej daty i status nie został wcześniej

zaktualizowany. Funkcja jest wywoływana jednorazowo przy uruchomieniu aplikacji, a następnie wykonywana automatycznie co minutę zgodnie z ustawionym harmonogramem. W przypadku wystąpienia błędu odpowiednia informacja jest zapisywana w konsoli.

- **Dodatkowe funkcje:**

- Panel admin: zabezpieczenie przed dodawaniem filmu w trakcie trwania innego filmu

```
router.post('/on_screen', authenticateToken, adminOnly, async (req, res) => {
  const { movie_id, screen_id, start_time, price } = req.body;
  if (!movie_id || !screen_id || !start_time || !price) {
    return res.status(400).json({ success: false, message: 'Brak wymaganych danych'
  });
}

  try {
    const conn = await getConnection();

    // Pobierz czas trwania filmu w minutach
    const [movieRows] = await conn.execute(
      'SELECT duration_minutes FROM movies WHERE id = ?',
      [movie_id]
    );
    if (movieRows.length === 0) {
      await conn.end();
      return res.status(404).json({ success: false, message: 'Film nie istnieje' });
    }
    const durationMinutes = movieRows[0].duration_minutes;
    const newStart = new Date(start_time);
    const newEnd = new Date(newStart.getTime() + durationMinutes * 60 * 1000);

    // Pobranie wszystkich seansów w tej samej sali
    const [existing] = await conn.execute(
      'SELECT id, start_time, movie_id FROM on_screen WHERE screen_id = ? AND
status = ?',
      [screen_id, 'upcoming']
    );

    for (let show of existing) {
      // Pobierz czas trwania istniejącego filmu
      const [existingMovie] = await conn.execute(
        'SELECT duration_minutes FROM movies WHERE id = ?',
        [show.movie_id]
      );
      const existingDuration = existingMovie[0].duration_minutes;
```

```

const existingStart = new Date(show.start_time);
const existingEnd = new Date(existingStart.getTime() + existingDuration * 60 *
1000);

if (newStart < existingEnd && newEnd > existingStart) {
  await conn.end();
  return res.status(400).json({
    success: false,
    message: 'Wybrana sala jest już zajęta w tym czasie.'
  });
}
}

```

Rys 16. Zabezpieczenie przed dodawaniem filmu w trakcie trwania innego filmu

- Nie można zalogowania w gdy ktoś inny zalogowany na tym samym koncie.

```

// Sprawdzamy, czy istnieje aktywny token w bazie
if (user.session_token) {
  try {
    jwt.verify(user.session_token, JWT_SECRET);
    // Token ważny => odrzuć logowanie
    await conn.end();
    return res.status(403).json({ success: false, message: 'Konto jest już zalogowane
na innym urządzeniu.' });
  } catch (err) {
    // Token wygasł => kontynuujemy logowanie
  }
}

// Tworzymy nowy token
const token = jwt.sign({ id: user.id, username: user.username, role: user.role },
JWT_SECRET, { expiresIn: '1h' });

// Zapis nowego tokena w bazie
await conn.execute('UPDATE users SET session_token = ? WHERE id = ?', [token,
user.id]);
await conn.end();

```

Rys 17. Zabezpieczenie przed logowaniem na wielu urządzeniach

- Odporność systemu na SQL Injection – przez zastosowanie zapytań parametryzowanych w których dane wejściowe użytkownika przekazywane są jako parametry, a nie jako część zapytania SQL. Dzięki temu wprowadzone dane są interpretowane wyłącznie jako wartości, a nie jako kod wykonywalny.

```

router.post('/login', async (req, res) => {

```

```

try {
  const { uname, psw } = req.body;
  if (!uname || !psw) return res.status(400).json({ success: false, message: 'Wszystkie pola są wymagane.' });

  const conn = await getConnection();
  const [rows] = await conn.execute(
    'SELECT id, username, password, role, session_token FROM users WHERE LOWER(username)=LOWER(?)',
    [uname]
  );

```

Rys 18. Przykład zapytań parametryzowanych

- W celu obsługi dynamicznych żądań dotyczących konkretnych seansów, zaimplementowano mechanizm **Route Parameters**. Wykorzystanie zapisu `:onScreenId` w definicji tras API pozwala na przekazywanie unikalnych identyfikatorów bezpośrednio w adresie URL. Wartości te są ekstrahowane za pomocą obiektu `req.params`, a następnie wykorzystywane w parametryzowanych zapytaniach SQL (Prepared Statements). Takie podejście zapewnia wysoką elastyczność systemu – jedna zdefiniowana trasa może obsłużyć tysiące różnych seansów – przy jednoczesnym zachowaniu pełnego bezpieczeństwa bazy danych przed atakami typu SQL Injection.”

```

router.get("/seats/:onScreenId", async (req, res) => {
  const id = req.params.onScreenId;

  try {
    const conn = await getConnection();

    const [seats] = await conn.execute(`
      SELECT ss.id, ss.seat_row, ss.seat_number,
      CASE WHEN r.id IS NOT NULL THEN 1 ELSE 0 END AS
taken
      FROM screen_seats ss
      JOIN on_screen os ON os.screen_id = ss.screen_id
      LEFT JOIN reservations r
        ON r.seat_id = ss.id AND r.on_screen_id = os.id
      WHERE os.id = ?
      ORDER BY ss.seat_row, ss.seat_number
    `, [id]);

    res.json({ success: true, seats });
    console.log("Fetching seats for onScreenId:", id);
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false });
  }

```

```
}  
});
```

Rys 19. Przykład pobierania danego id przez URL

2.3. Frontend (Interfejs użytkownika)

- **HTML5 & CSS3**

- **Zastosowanie:** Budowa struktury strony i jej warstwa wizualna. Zastosowano podział na dedykowane arkusze stylów dla różnych modułów (np. osobne style dla panelu admina i widoku rezerwacji).
- **Lokalizacja:** frontend/html/, frontend/css/.

```
<!DOCTYPE html>  
<html lang="pl_PL">  
<head>  
  <meta charset="UTF-8">  
  <title>ETIKINO</title>  
  <meta name="keywords" content="kino,eti,ETI,internet,biblioteka filmowa">  
  <meta name="description" content="Cinema App">  
  <meta name="author" content="Maślanka Szymon">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"  
rel="stylesheet" integrity="sha384-  
T3c6Coli6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN"  
crossorigin="anonymous">  
  <link href="/css/index.css" rel="stylesheet">  
  <link rel="icon" type="image/x-icon" href="/img/logo.ico">  
  
</head>  
<body>  
  
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"  
integrity="sha384-  
FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"  
crossorigin="anonymous"></script>  
  
<div id="regNotification" class="notification"></div>  
  
<header class="pc-header">  
  <div class="container-fluid headercolor">  
    <div class="d-flex justify-content-center align-items-center gap-4 flex-wrap">  
        
      <h1 class="headertext1 mb-0">Kino</h1>  
      <div class="col justify-content-start" id="listbtns">  
        <ol class="listag d-flex align-items-center mb-0" id="listbtns">
```



```

        <li><a href="#" id="libraryLink">Biblioteka</a></li>
        <li><button onclick="toggleForm('login')" id="logbutton">Zaloguj</button></li>
        <li><button onclick="toggleForm('register')"
id="regbutton">Zarejestruj</button></li>
    </ol>
</div>
</div>
</div>
<div class="infobar">
    <div class="container-fluid text-center justify-content-center">
        <strong>ETIKINO</strong> — najlepsze kino w Trójmieście • Czynne codziennie
od 9:00 do 23:00 • Tel: <strong>123 456 789</strong>
    </div>
</div>
</header>

<header class="mobile-header">
    <div class="m-header-main">
        <div class="m-left">
            
            <h1 class="m-title">Kino</h1>
        </div>

        <button class="hamburger" id="hamburger">≡</button>
    </div>

    <nav class="m-nav" id="m-nav">
        <a href="#" id="libraryLinkMobile" class="m-link">Biblioteka</a>
        <button onclick="toggleForm('login')" class="m-btn-log">Zaloguj</button>
        <button onclick="toggleForm('register')" class="m-btn-reg">Zarejestruj</button>
    </nav>

    <div class="m-infobar">
        <strong>ETIKINO</strong> — najlepsze kino w Trójmieście • 9:00–23:00 • Tel:
<strong>123 456 789</strong>
    </div>
</header>

<div id="carouselExampleIndicators" class="carousel slide">
    <div class="carousel-indicators">
        <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-
to="0" class="active" aria-current="true" aria-label="Slide 1"></button>
        <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-
to="1" aria-label="Slide 2"></button>
    </div>

```

```

    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-
to="2" aria-label="Slide 3"></button>
</div>
<div class="carousel-inner">
    <div class="carousel-item active">
        
    </div>
    <div class="carousel-item">
        
    </div>
    <div class="carousel-item">
        
    </div>
</div>
<button class="carousel-control-prev" type="button" data-bs-
target="#carouselExampleIndicators" data-bs-slide="prev">
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Previous</span>
</button>
<button class="carousel-control-next" type="button" data-bs-
target="#carouselExampleIndicators" data-bs-slide="next">
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Next</span>
</button>
</div>

<section class="film-carousel my-5">
    <div class="container-fluid sectioncolor1 mb-2 mt-4">
        <div class="row align-items-center">
            <div class="col">
                <h1 class="text-start">Na ekranie</h1>
            </div>
        </div>
    </div>
    <div class="carousel-container">
        <button class="carousel-btn prev"></button>
        <div class="carousel-track" id="now_playing"></div>
    </div>

```

```

    <button class="carousel-btn next">></button>
  </div>
</section>

<section class="film-carousel my-5">
  <div class="container-fluid sectioncolor1 mb-2 mt-4">
    <div class="row align-items-center">
      <div class="col">
        <h1 class="text-start">Wkrótce</h1>
      </div>
    </div>
    <div class="carousel-container">
      <button class="carousel-btn prev"><<</button>
      <div class="carousel-track" id="coming_soon"></div>
      <button class="carousel-btn next">>></button>
    </div>
  </section>

<section class="film-carousel my-5">
  <div class="container-fluid sectioncolor1 mb-2 mt-4">
    <div class="row align-items-center">
      <div class="col">
        <h1 class="text-start">Dla rodzin</h1>
      </div>
    </div>
    <div class="carousel-container">
      <button class="carousel-btn prev"><<</button>
      <div class="carousel-track" id="family"></div>
      <button class="carousel-btn next">>></button>
    </div>
  </section>

<section class="film-carousel my-5">
  <div class="container-fluid sectioncolor1 mb-2 mt-4">
    <div class="row align-items-center">
      <div class="col">
        <h1 class="text-start">Akcji</h1>
      </div>
    </div>
    <div class="carousel-container">
      <button class="carousel-btn prev"><<</button>
      <div class="carousel-track" id="action"></div>

```

```

    <button class="carousel-btn next">></button>
  </div>
</section>

<section class="sekcja3">
  <div class="container-fluid d-flex justify-content-center mb-20">
    <button type="button" class="btn btn-primary mx-auto buttonlos"
onclick="zmien()">Kup bilet</button>
  </div>
</section>

<div id="login" style="display:none;">
<form id="loginForm" onsubmit="return false;">
  <h2>Logowanie</h2>

  <div class="form-group">
    <label for="loginUname">Nazwa użytkownika</label>
    <input type="text" name="uname" id="loginUname" placeholder="Wpisz nazwę
użytkownika">
  </div>

  <div class="form-group">
    <label for="loginPsw">Hasło</label>
    <input type="password" name="psw" id="loginPsw" placeholder="Wpisz hasło">
  </div>
  <div id="loginError"></div>
  <button type="button" id="loginBtn">Zaloguj się</button>

</form>

<div class="cancelbtn">
  <button type="button"
onclick="document.getElementById('login').style.display='none'">Anuluj</button>
</div>
</div>

<div id="register" style="display:none;">
<form id="registerForm">
  <h2>Rejestracja</h2>

  <div class="form-group">
    <label for="uname">Nazwa użytkownika</label>
    <input type="text" name="uname" id="uname" placeholder="Wpisz nazwę
użytkownika">
  </div>

```

```

<div class="form-group">
  <label for="email">Adres e-mail</label>
  <input type="email" name="email" id="email" placeholder="Wpisz adres e-mail">
</div>

<div class="form-group">
  <label for="psw">Hasło</label>
  <input type="password" name="psw" id="psw" placeholder="Wpisz hasło">
</div>

<div class="form-group">
  <label for="pswConfirm">Powtórz hasło</label>
  <input type="password" name="pswConfirm" id="pswConfirm" placeholder="Powtórz
hasło">
</div>
<div id="regError"></div>
<button type="button" id="registerBtn">Zarejestruj</button>

</form>

<div class="cancelbtn">
  <button type="button"
onclick="document.getElementById('register').style.display='none'">Anuluj</button>
</div>
</div>

<script src="/js/index.js"></script>
</body>
</html>

```

Rys 20. Plik index.html (dla przykładu technologii)

Zastosowane technologie html i css:

- semantyczne znaczniki: <header>, <section>, <nav>, <div>, <form>, <h1>–<h2> dla struktury strony.
- Meta dane w <head> – charset, viewport, keywords, description, author dla SEO i responsywności.
- Łączenie stylów i skryptów:
 - <link> – do CSS (Bootstrap i własny index.css).
 - <script> – do JavaScript (Bootstrap JS i własny index.js).
- Framework Bootstrap 5:
 - Siatka (grid) i kolumny dla układu strony.
 - Gotowe komponenty: karuzela (carousel), przyciski (btn), nagłówki responsywne.
- Identyfikatory (id) i klasy (class) – do stylizacji CSS i manipulacji w JS (#login, .carousel-track, .btn, .sectioncolor1).

- Formularze interaktywne – <form> z polami <input> i przyciskami <button>, obsługiwane przez JS, w tym walidacja po stronie klienta.
- Dynamiczne elementy strony – kontenery do karuzel filmowych (#now_playing, #coming_soon, #family, #action) wypełniane danymi z backendu przy użyciu JS.
- Responsywność – osobne nagłówki i nawigacja dla desktop i mobile, wykorzystanie Bootstrap i CSS do adaptacji wyglądu.
 - box-sizing: border-box – kontrola wymiarów elementów uwzględniająca padding i border.
 - display: flex i flex-direction – elastyczne układy dla nagłówków, przycisków i karuzel.
 - gap, justify-content, align-items – rozmieszczenie elementów w kontenerach flex.
 - position: fixed, absolute, relative – pozycjonowanie nagłówków, formularzy, przycisków i powiadomień.
 - z-index – warstwowanie elementów (formularze, powiadomienia, karuzela, nagłówki).
- Transformacje i animacje
 - transform – skalowanie elementów (scale) przy hover lub animacjach karuzeli.
 - translate(-50%, -50%) – centrowanie elementów (formularze, powiadomienia).
 - transition – płynne zmiany właściwości (color, background, transform) przy hover.
 - @keyframes – animacje np. fadeIn nagłówków, fadeInOutNotification dla powiadomień.
- Kolory, gradienty i cienie
 - background: linear-gradient() – gradienty w nagłówkach, przyciskach i karuzelach.
 - box-shadow – cienie dla nagłówków, formularzy, przycisków i powiadomień.
 - text-shadow – efekt cienia dla nagłówków i tekstu w nagłówkach.
- Typografia
 - @font-face – własna czcionka (Pixar.ttf).
 - font-family, font-weight, letter-spacing – kontrola wyglądu tekstu.
 - font-size w vw i vh – skalowanie tekstu responsywne do rozmiaru ekranu.
 - color – kontrastowy tekst na ciemnym tle.
- Formularze i przyciski
 - Stylizacja pól formularzy: border-radius, padding, background-color, color.
 - Placeholder styling: ::placeholder.
 - Focus efekty: :focus, box-shadow dla podświetlenia pola.
 - Przycisk: hover, active, transformacje (scale) i gradienty.
- Responsywność – dla strony głównej
 - @media queries – zmiany rozmiarów, układu, font-size, karuzel i nagłówków dla tabletów i telefonów.
 - Zmiana display w zależności od rozdzielczości (mobile vs desktop).
 - Skalowanie karuzeli i przycisków przy różnych szerokościach ekranu.
 - Przełączanie nagłówków (.pc-header / .mobile-header).

- Zmiana układu przycisków, tekstów i formularzy w zależności od szerokości ekranu.
- Zmiana rozmiarów elementów w vw/vh zamiast px dla płynnego skalowania.
- Responsywne menu: hamburger menu dla urządzeń mobilnych.
- Karuzele i przewijanie
 - overflow-x: auto, scroll-behavior: smooth – płynne przewijanie poziome w karuzelach.
 - Ukrywanie scrollbar: ::-webkit-scrollbar { display: none; } i scrollbar-width: none.
 - flex: 0 0 auto – elementy karuzeli zachowują swoją szerokość przy scrollowaniu.
- Efekty interaktywne
 - hover – zmiana koloru, tła, cieni, transformacji.
 - Powiadomienia: display: none/block i animacje fade in/out.
 - Skalowanie i animacja zdjęć w karuzeli (transform: scale, transition).
- Przezroczystość i nakładki
 - rgba() – półprzezroczyste tła dla infobar i nakładki gradientowej na karuzeli.
 - pseudo-elementy – np. ::after
- **JavaScript (Client-side)**
 - **Zastosowanie:** Obsługa interakcji z użytkownikiem (dynamiczne wybieranie miejsc na sali, walidacja formularzy) oraz komunikacja z backendem za pomocą fetch/axios.
 - **Lokalizacja:** frontend/js/.
 - **Technologie:**

Zmiana client.html

- Strona client.html zależnie czy zalogowano na admin czy user zmienia swój wygląd dodając odpowiednie ustawienia w rozwijanym menu.

```
if(role === 'admin') {
  document.querySelectorAll('.admin-only').forEach(el => el.style.display = 'block');
} else {
  document.querySelectorAll('.user-only').forEach(el => el.style.display = 'block');
}

//Ustawienia konta
const role = getUserRoleFromToken();
document.addEventListener('DOMContentLoaded', () => {
  const settingsBtn = document.getElementById('openSettingsClient');
  if (settingsBtn && role === 'user') {
    settingsBtn.addEventListener('click', (e) => {
      e.preventDefault();
      window.location.href = 'client_settings.html';
    });
  }
});
```

```

    });
  }
});
document.addEventListener('DOMContentLoaded', () => {
  const settingsBtn = document.getElementById('openSettingsAdmin');
  if (settingsBtn && role === 'admin') {
    settingsBtn.addEventListener('click', (e) => {
      e.preventDefault();
      window.location.href = 'admin_panel.html';
    });
  }
});
});

```

Rys 22. Elementy obsługujące zmianę client.html zależnie od zalogowanego użytkownika lub admina

Manipulacja DOM

- document.getElementById() – pobieranie elementów po ID.
- document.querySelector() / querySelectorAll() – pobieranie elementów po selektorze CSS.
- element.style.property – zmiana stylów inline (np. display, backgroundColor).
- element.classList.add() / remove() / toggle() – dodawanie, usuwanie i przełączanie klas CSS.
- element.appendChild() – dodawanie nowych elementów do DOM.
- element.textContent – ustawianie lub pobieranie tekstu elementu.
- element.addEventListener() – obsługa zdarzeń (click, DOMContentLoaded itp.).

Formularze i walidacja

- Odczyt wartości pól: input.value i trim().
- Prosta walidacja: porównanie haseł (psw !== pswConfirm).
- form.reset() – czyszczenie formularza.
- Wyświetlanie komunikatów błędów w dedykowanych elementach (errorBox.textContent).

Powiadomienia

- Dynamiczne pokazywanie/ukrywanie powiadomień (notification.classList.add('show')).
- setTimeout() – automatyczne ukrywanie powiadomienia po określonym czasie.
- Ustawianie koloru powiadomień w zależności od typu (success, warning).

Przechowywanie tokenów / autoryzacja

- localStorage.setItem() / getItem() / removeItem() – przechowywanie tokenu JWT w przeglądarce.
- Funkcje pomocnicze:
 - saveToken(token), getToken(), removeToken() – zarządzanie tokenem.

- `authHeaders()` – generowanie nagłówków autoryzacji z tokenem.
- `isTokenExpired(token)` – sprawdzanie wygaśnięcia tokenu (dekodowanie JWT).

Fetch API / komunikacja z backendem

- `fetch(url, opts)` – wysyłanie zapytań HTTP (GET, POST).
- `await response.json()` – parsowanie odpowiedzi JSON.
- Obsługa błędów z `try/catch`.
- `safeFetch()` – funkcja opakująca `fetch`, automatycznie dodająca token do nagłówków.

Nawigacja i przekierowania

- `window.location.href` / `window.location.replace()` – zmiana strony.
- Zapobieganie domyślnemu zachowaniu linków: `event.preventDefault()`.

Karuzele / przewijanie

- `scrollBy({ left: X, behavior: 'smooth' })` – przewijanie poziome elementów karuzeli.
- Dynamiczne obliczanie szerokości slajdów: `slide.offsetWidth + parseInt(getComputedStyle(track).gap)`.
- Obsługa przycisków poprzedni/następny dla każdej sekcji.

Interakcje UI

- `toggleForm(formId)` – prosty mechanizm przełączania widoczności formularzy logowania/rejestracji.
- Hamburger menu dla mobilnego nagłówka: `nav.classList.toggle('active')`.
- Ukrywanie / pokazywanie elementów w zależności od stanu użytkownika (np. tokenu).

Inne techniki

- Event `DOMContentLoaded` – inicjalizacja skryptów dopiero po załadowaniu DOM.
- Dynamiczne generowanie elementów (`document.createElement('div')`, `img.src`, `img.alt`) dla sekcji filmów.
- Użycie `console.log` do debugowania (`console.log`, `console.warn`, `console.error`).
- Prosta logika biznesowa dla sekcji filmów (przypisanie do odpowiedniego kontenera wg gatunku i daty).

2.4. Bezpieczeństwo i Konfiguracja

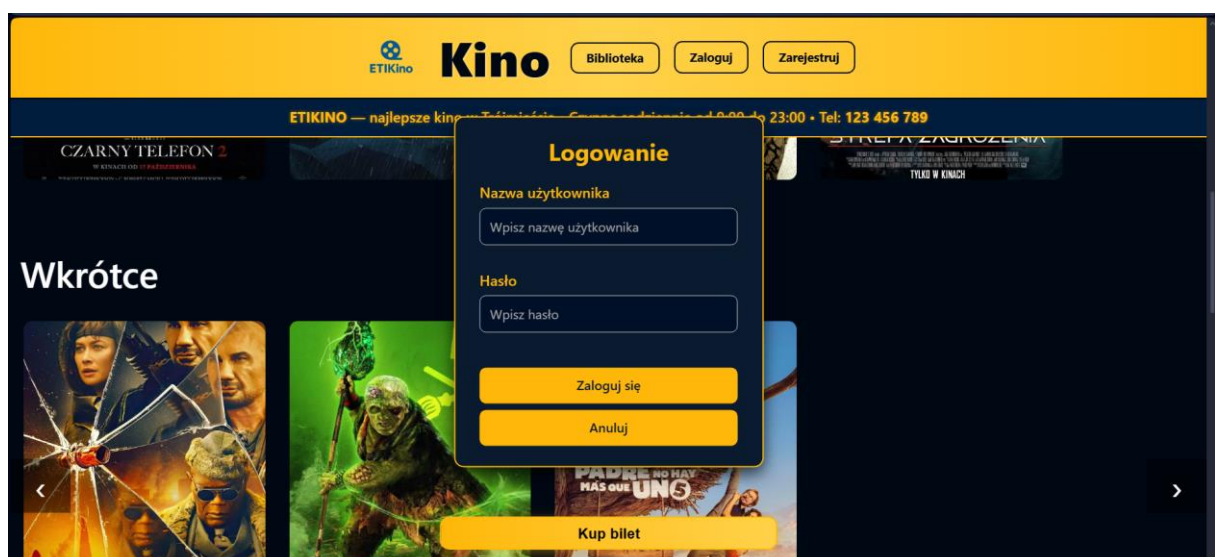
- **Dotenv (.env)**

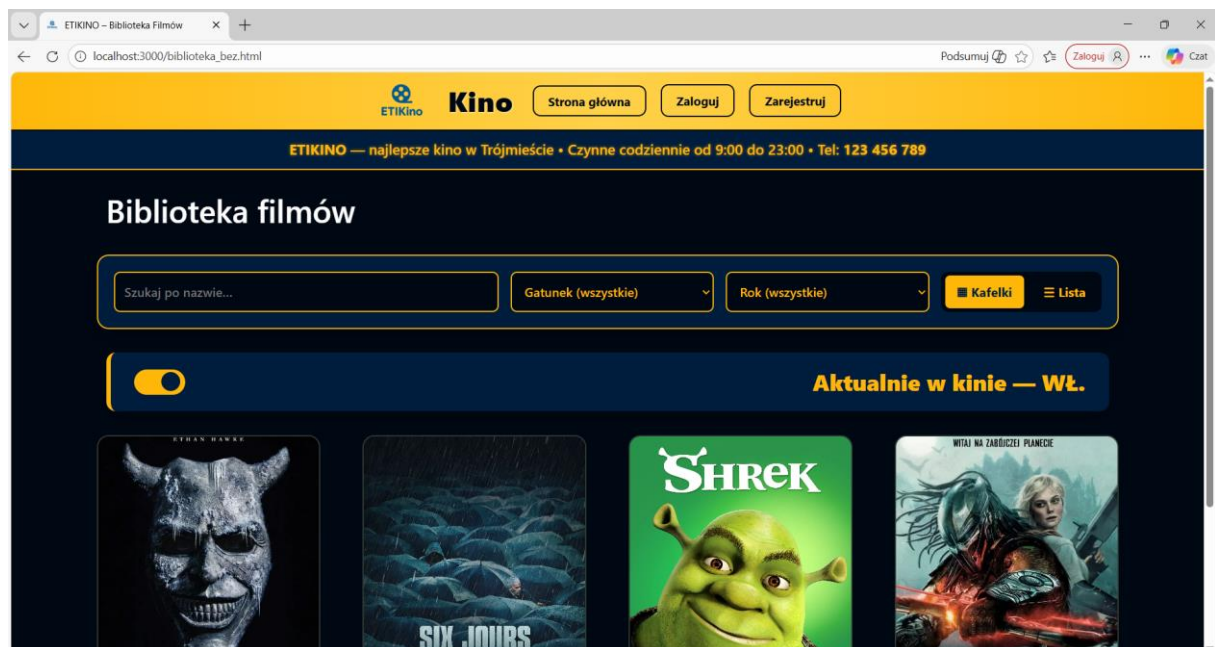
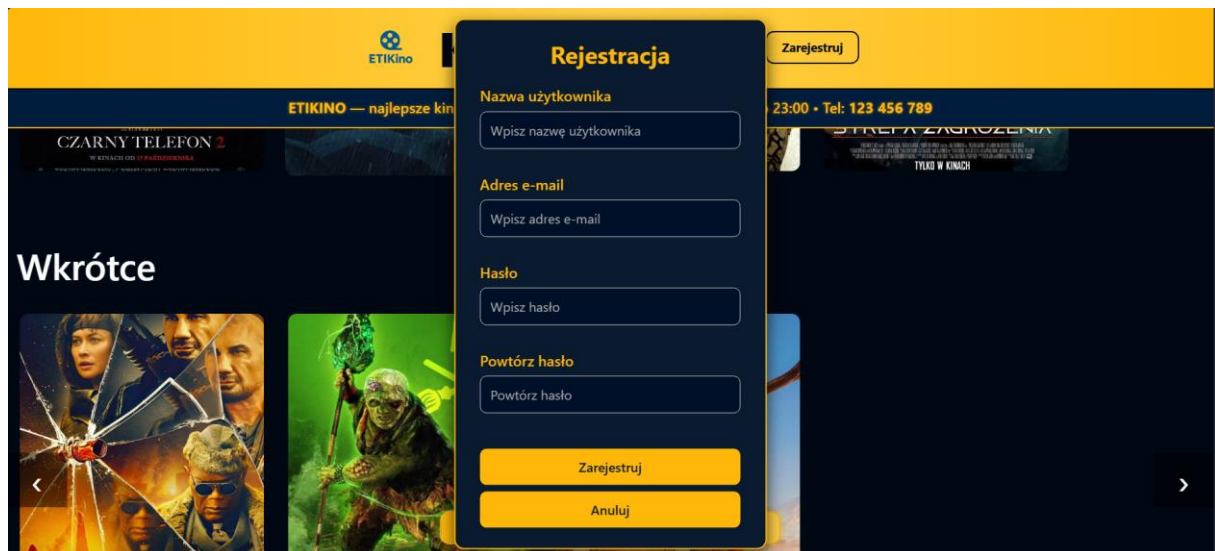
- **Zastosowanie:** Zarządzanie zmiennymi środowiskowymi. Przechowuje wrażliwe dane (hasło do bazy, klucz JWT, port serwera), które nie powinny być widoczne bezpośrednio w kodzie źródłowym.
- **Lokalizacja:** backend/.env.

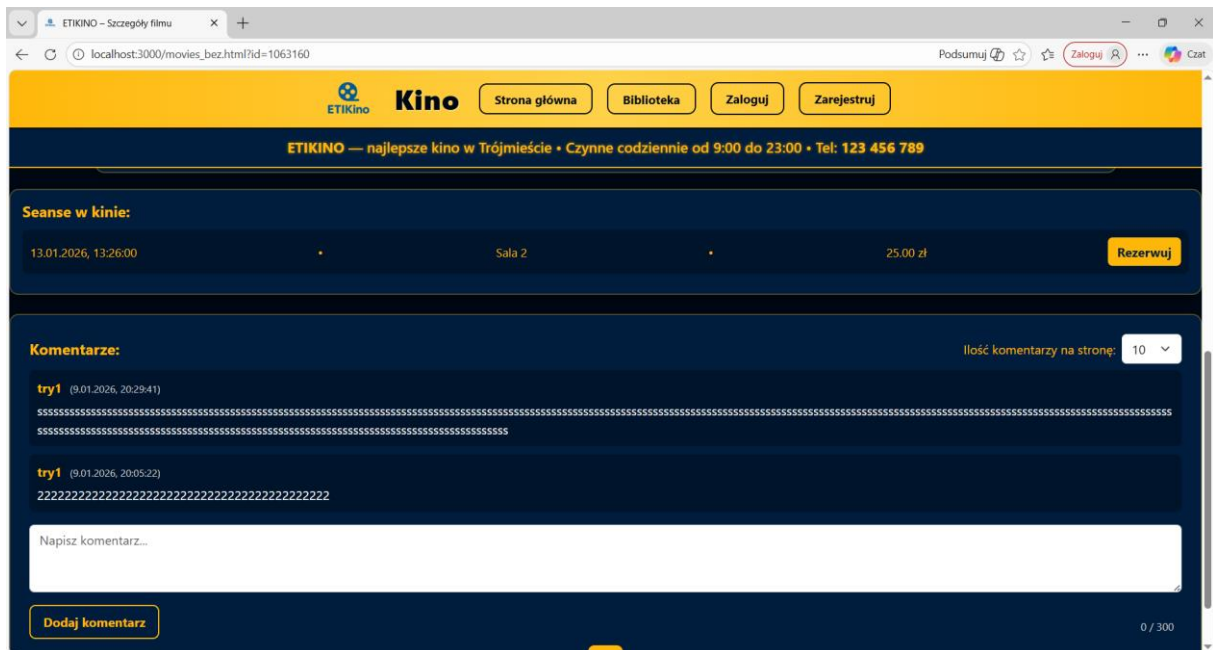
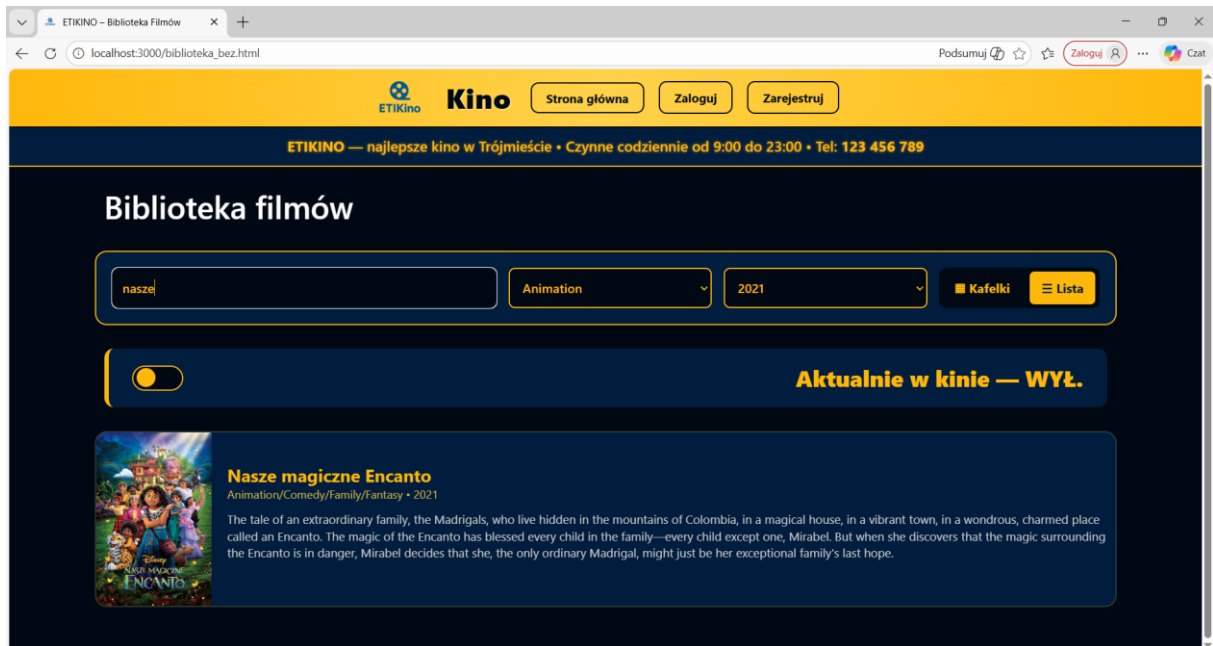
```
JWT_SECRET=tajny_klucz_etikino_jwt
PORT=3000
HOST = localhost
USER = root
PASSWORD = ""
DATABASE = kino_eti
```

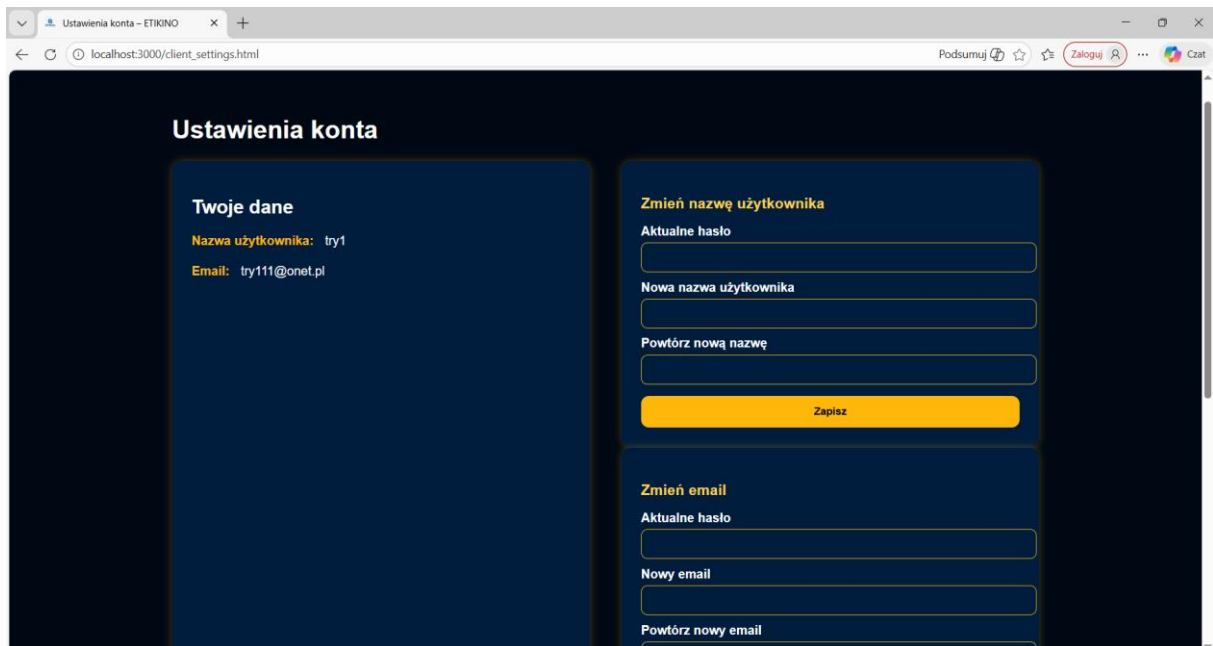
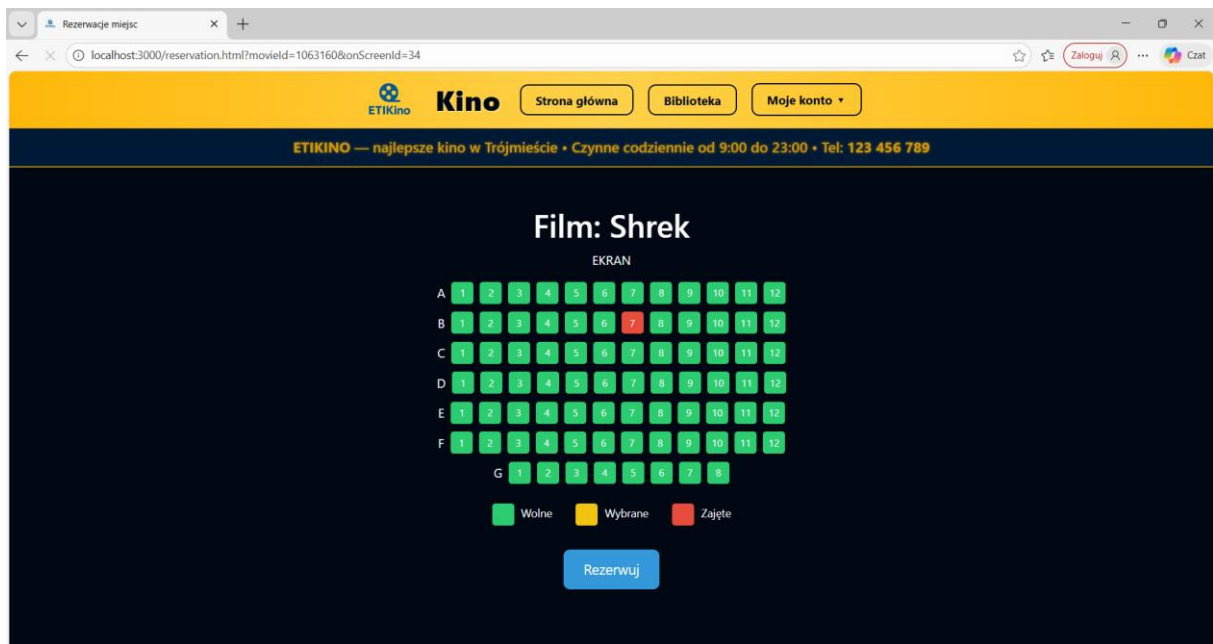
Rys 23. Plik .env

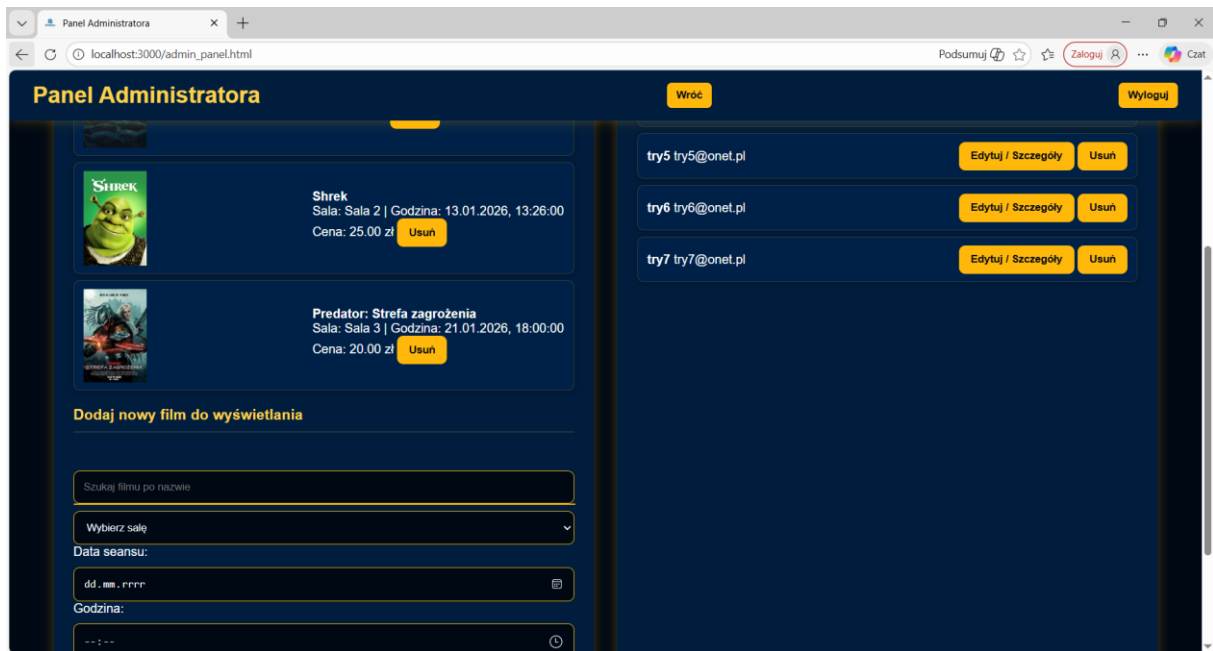
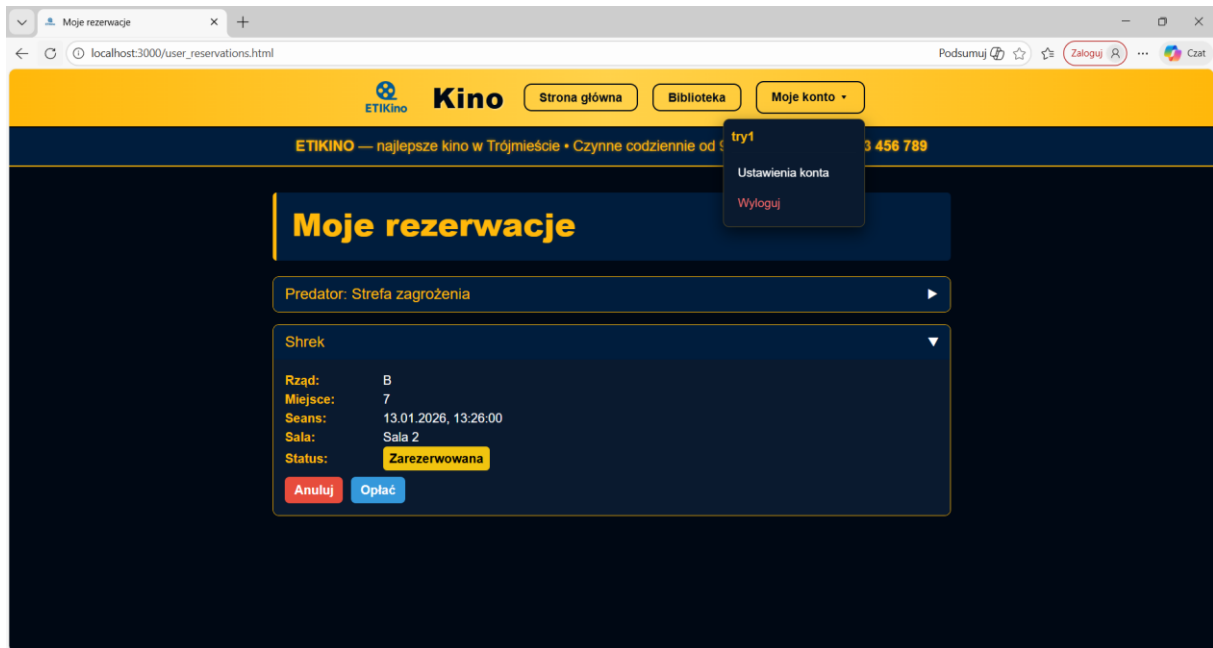
3. Pokaz działania aplikacji











Szczegóły użytkownika

localhost:3000/user_edit.html?id=16

Szczegóły użytkownika

Dane użytkownika

Nazwa użytkownika

ky1

Email

ky111@onet.pl

Nowe hasło (opcjonalnie)

Zapisz zmiany

Komentarze

Predator: Strefa zagrożenia

Shrek

Rezerwacje

RESERVED

Shrek

Sala: Sala 2

Miejsce: B7

13.01.2026, 13:28:00

Usuń

Opłać

PAID

ENDED

 **Kino**



Biblioteka

Zaloguj

Zarejestruj

ETIKINO — najlepsze kino w Trójmieście •

9:00–23:00 • Tel: 123 456 789

Na ekranie

Kup bilet

4. Wnioski

Realizacja projektu „Kino ETI” pozwoliła na praktyczne zweryfikowanie wiedzy z zakresu projektowania nowoczesnych systemów webowych. Zastosowanie mechanizmu Express Router umożliwiło logiczne odseparowanie modułów autoryzacji, zarządzania repertuarem oraz rezerwacji. Taka struktura znacząco ułatwia debugowanie kodu i pozwala na skalowanie aplikacji bez wprowadzania chaosu w głównym pliku serwera. Wykorzystanie modelu Async/Await oraz biblioteki Axios okazało się niezbędne dla zachowania płynności interfejsu (UX). Dzięki asynchroniczności operacje wejścia/wyjścia (I/O), takie jak zapytania do bazy MariaDB czy komunikacja z API TMDb, nie blokują wątku głównego serwera, co pozwala na jednoczesną obsługę wielu użytkowników. Implementacja standardu JWT (JSON Web Token) zapewniła bezpieczną, bezstanową komunikację. Dodatkowo, zastosowanie wyrażeń regularnych (Regex) oraz biblioteki Validator na etapie rejestracji pozwoliło na skuteczną eliminację błędnych danych i wymuszenie silnych haseł jeszcze przed ich zahashowaniem algorytmem bcrypt. Zastosowanie dwupoziomowego systemu aktualizacji statusów — globalnego zadania Cron (działającego w tle) oraz lokalnego mechanizmu Lazy Update (wyzwalanego przez req.params w trasach użytkownika) — zagwarantowało wysoką integralność danych. System jest odporny na opóźnienia i zawsze prezentuje użytkownikowi stan faktyczny zajętości miejsc. Integracja z API The Movie Database (TMDb) pokazała, jak skutecznie zasilać autorskie systemy profesjonalnymi danymi masowymi. Zaimplementowany mechanizm Throttlingu (opóźnienie delay) pozwolił na stabilne pobranie tysięcy rekordów przy zachowaniu limitów narzuconych przez dostawcę usługi. Użycie mechanizmu req.params pozwoliło na stworzenie uniwersalnych punktów końcowych (endpoints), które dynamicznie dostosowują treść do identyfikatora w URL. Jest to rozwiązanie zgodne ze standardem REST API, które ułatwia nawigację i pozwala na poprawne indeksowanie zasobów. Projekt udowodnił, że połączenie technologii Node.js, Express i MySQL pozwala na stworzenie wydajnego i bezpiecznego systemu rezerwacyjnego. Zdobyte doświadczenie w zakresie integracji frontendu z backendem oraz zarządzania cyklem życia danych w bazie relacyjnej stanowi solidny fundament do budowy bardziej złożonych systemów informatycznych w przyszłości.